



인프라 취약점 진단

신해진

목차

- 셸스크립트 개선
 - Linux
 - DBMS
 - Kubernetes
 - Docker
- 질의서 작성
 - 클라우드 관리체계
 - 클라우드 관리콘솔
 - AWS
 - Azure

• LINUX SRV-001 & SRV-176 “안전한 네트워크 모니터링 서비스 사용” ShellScript 개선 전후 스크립트 실행 결과

항목 코드	스크립트 실행 결과 (24년도)	스크립트 실행 결과 (25년도)
LINUX <u>SRV-001/176</u>	<<u>자산명</u>> <u>자산명</u> </<u>자산명</u>> <<u>IP주소</u>> 172.16.X.X </<u>IP주소</u>> <<u>플랫폼</u>> OS </<u>플랫폼</u>> <<u>소프트웨어</u>> 리눅스 </<u>소프트웨어</u>> <<u>점검항목</u>> <u>SRV-001</u> </<u>점검항목</u>> <<u>점검항목명</u>> <u>SNMP Community String 설정 미흡</u> </<u>점검항목명</u>> <점검 결과>Y</점검 결과> <점검 내용> <u>SNMP Community String</u>이 Default 설정이 아니므로 양호함 [현황] <u>/etc/snmp/snmpd.conf</u> 파일 설정 <u>com2sec notConfigUser default EnGYIFEInc</u> <u>root 00000 1 0 00:00 ? 00:00:00 /usr/sbin/snmpd L50 6d -f</u> </점검 내용>	<<u>자산명</u>> <u>자산명</u> </<u>자산명</u>> <<u>IP주소</u>> 172.16.X.X </<u>IP주소</u>> <<u>플랫폼</u>> OS </<u>플랫폼</u>> <<u>소프트웨어</u>> 리눅스 </<u>소프트웨어</u>> <<u>점검항목</u>> <u>SRV-001</u> </<u>점검항목</u>> <<u>점검항목명</u>> <u>안전한 네트워크 모니터링 서비스 사용</u> </<u>점검항목명</u>> <점검 결과>Y</점검 결과> <점검 내용> <u>SNMP Community String</u>이 Default 설정이 아니므로 양호함 <u>(SNMP V3 버전 사용하고 있음)</u> [현황] <u>com2sec notConfigUser default EnGYIFEInc</u> </점검 내용>

• LINUX SRV-001 & SRV-176 “안전한 네트워크 모니터링 서비스 사용” 스크립트 수정 내역

항목 코드	스크립트 수정 전 (24년도)	스크립트 수정 후 (25년도)
<u>LINUX</u> <u>SRV-001/176</u>	1) <u>SRV-001</u> 점검 항목명 “ <u>SNMP Community String</u> 설정 미흡” <u>SRV-001</u> “ <u>SNMP Community String</u> ” <u>SRV-176</u> “취약한 <u>SNMP</u> 버전 사용”	1) <u>SRV-001</u> 점검 항목명을 “ <u>SNMP Community String</u> 설정 미흡” > “ <u>안전한 네트워크 모니터링 서비스 사용</u> ”으로 변경 <u>SRV-001</u> “ <u>안전한 네트워크 모니터링 서비스 사용</u> ”
	2) <u>SRV-176</u> 점검 항목 [현황]에 <u>SNMP v3</u> 설정 내역 출력	2) <u>SRV-001</u> , <u>SRV-176</u> 통합으로 <u>SNMPv3</u> 설정 내역을 <u>SRV-001</u> [현황]에 추가 및 <u>SRV-176</u> 항목 삭제 if [-f /etc/ <u>snmp/snmpd.conf</u>]; then if [\$(cat /etc/ <u>snmp/snmpd.conf</u> grep -E “^s*(<u>rouser</u> <u>rwuser</u>)s+” wc -l) -ge 1]; then Result=”Y” Exp=” <u>SNMP V3</u> 버전 사용하고 있음”
	3) 기본 생성되는 <u>snmp.conf</u> 파일 경로만 [현황]에 출력	3) RPM 명령어 이용하여 <u>snmp.conf</u> 파일을 다른 위치에서도 찾을 수 있도록 수정 <u>snmp_conf_path</u> = `rpm -ql net- <u>snmp</u>   grep <u>snmpd.conf</u>   grep -v “gz”`

- Linux SRV-001 “안전한 네트워크 모니터링 서비스 사용” 스크립트 비교

Shell ▾	복사캡션...	Shell ▾	복사캡션...
# 2024년 도 Linux SRV-001 "SNMP Community 스트링 설정 미흡" (Shellscript 개선 전)		# 2025년 도 Linux SRV-001 & 176 통합 "안전한 네트워크 모니터링 서비스 사용" (Shellscript 개선 후)	
<pre>snmp_community() { iCODE="SRV-001" iTitle="SNMP Community 스트링 설정 미흡" Item_head "\$iCODE" "\$iTitle" if [`ps -ef   grep snmp   grep -v "dmi"   grev -v "grep"   wc -l` -eq 0] then Result="Y" Exp="SNMP 프로세스가 비실행 중이므로 양호함" Evi="" else snmp="" snmpnum=0 if [-f /etc/snmpd.conf] then snmp=`echo "\$snmp";echo " /etc/snmpd.conf 파일 설정:"` snmp=`echo "\$snmp";cat /etc/snmpd.conf   egrep -i "public private com2sec community"   grep -v "^#"` snmpnum=1 fi if [-f /etc/snmp/snmpd.conf] then snmp=`echo "\$snmp";echo " /etc/snmp.snmpd.conf 파일 설정:"` snmp=`echo "\$snmp";cat /etc/snmp.snmpd.conf   egrep -i "public private com2sec community"   grep -v "^#"` snmpnum=1 fi if [-f /etc/snmp/conf/snmpd.conf] then snmp=`echo "\$snmp";echo " /etc/snmp.snmpd.conf 파일 설정:"` snmp=`echo "\$snmp";cat /etc/snmp.snmpd.conf   egrep -i "public private com2sec community"   grep -v "^#"` snmpnum=1 fi if [\$snmpnum -eq 1] then Result="Y" Exp="snmp 서비스는 동작중이지만 snmpd.conf 파일이 존재하지 않으므로 수동점검이 필요함" Evi=`ps -ef   grep snmp   grep -v "dmi"   grep -v "grep"` fi fi Item_foot "\$Result" "\$Exp" "\$Evi" }</pre>		<pre>snmp_monitoring() { iCODE="SRV-001" iTitle="안전한 네트워크 모니터링 서비스 상요" Item_head "\$iCODE" "\$iTitle" snmp_conf_path=`rpm -ql net-snmp   grep snmpd.conf   grep -v "gz"` # snmp가 있는지 r edhat의 rpm으로 query, list로 확인 if [`ps -ef   grep snmp   grep -v "dmi"   grep -v "grep"   wc -l` -eq 0] then Result="Y" Exp="SNMP 서비스를 사용하지 않으므로 양호함" Evi="" else if [-f \$snmp_conf_path] then if [` cat \$snmp_conf_path   grep -q -E "^s*(rouser rwuser) \s+"   wc -l` -eq 1] then Result="Y" Exp="SNMPv3를 사용 중으로 양호함(rouser rwuser) (snmp.conf path : \$snmp_conf_path)" Evi=`cat \$snmp_conf_path   grep -q -E "^s*(rouser rwuser)\s+"` else if [`cat \$snmp_conf_path   egrep -i "com2sec community"   egrep -v "public private"   grep -v "^#"   wc -l` -ge 1] then Result="Y" Exp="default snmp community string을 변경하여 운용하므로 양호함 (snmp.conf path : \$snmp_conf_path)" Evi=`cat \$snmp_conf_path   egrep -i "com2sec community"   grep -v "^s*(rouser rwuser) \s+"` fi fi else if [-f /etc/snmpd.conf] then if [`cat /etc/snmpd.conf   grep -q -E "^s*(rouser rwuser)\s+"   wc -l` -eq 1] then Result="Y" Exp="SNMPv3를 사용 중으로 양호함(rouser rwuser) (snmp.conf path : \$snmp_conf_path)" Evi=` cat /etc/snmpd.conf   grep -q -E "^s*(rouser rwuser)\s+"` else if [`cat /etc/snmpd.conf   egrep -i "com2sec community"   egrep -v "public private"   grep -v "^#"   wc -l` -ge 1] then Result="Y" Exp="default snmp community string을 변경하여 운용하므로 양호함" Evi=`cat /etc/snmpd.conf   egrep -i "com2sec community"   grep -v "^s*(rouser rwuser) \s+"` else Result="N" Exp="default snmp community string을 변경하여 사용하지 않으므로 취약함 (snmp.conf path : /etc/snmpd.conf)" Evi=`cat /etc/snmpd.conf   egrep -i "com2sec community"   grep -v "^s*(rouser rwuser) \s+"` fi fi elif [-f /etc/snmp/snmpd.conf]; then if [\$(cat /etc/snmp/snmpd.conf grep -E "^s*(rouser rwuser)\s+" wc -l) -ge 1]; then Result="Y" Exp="SNMPv3를 사용 중으로 양호함(rouser rwuser) (snmp.conf path : /etc/snmp.conf)" Evi=`cat /etc/snmp/snmpd.conf   grep -q -E "^s*(rouser rwuser)\s+"` else if [`cat /etc/snmpd.conf   egrep -i "com2sec community"   egrep -v "public private"   grep -v "^#"   wc -l` -ge 1] then Result="Y" Exp="default snmp community string을 변경하여 운용하므로 양호함" Evi=`cat /etc/snmpd.conf   egrep -i "com2sec community"   grep -v "^#"` else Result="N" Exp="default snmp community string을 변경하여 사용하지 않으므로 취약함 (snmp.conf path : /etc/snmpd.conf)" Evi=`cat /etc/snmpd.conf   egrep -i "com2sec community"   grep -v "^#"` fi fi fi fi fi }</pre>	

• LINUX SRV-028 “원격 터미널 접속 타임아웃 설정 미흡” ShellScript 개선 전후 스크립트 실행 결과

항목 코드	스크립트 실행 결과 (24년도)	스크립트 실행 결과 (25년도)
<u>LINUX</u> <u>SRV-028</u>	<<u>자산명</u>> <u>자산명</u> </<u>자산명</u>> <<u>IP주소</u>> 172.16.X.X </<u>IP주소</u>> <<u>플랫폼</u>> OS </<u>플랫폼</u>> <<u>소프트웨어</u>> 리눅스 </<u>소프트웨어</u>> <<u>점검항목</u>> <u>SRV-028</u> </<u>점검항목</u>> <<u>점검항목명</u>> <u>원격 터미널 접속 타임아웃 설정 미흡</u> </<u>점검항목명</u>> <<u>점검 결과</u>>Y</<u>점검 결과</u>> <<u>점검 내용</u>> <u>TMOUT</u> 설정이 600 이하로 설정되어 있으므로 양호함 [현황] </<u>점검 내용</u>>	<<u>자산명</u>> <u>자산명</u> </<u>자산명</u>> <<u>IP주소</u>> 172.16.X.X </<u>IP주소</u>> <<u>플랫폼</u>> OS </<u>플랫폼</u>> <<u>소프트웨어</u>> 리눅스 </<u>소프트웨어</u>> <<u>점검항목</u>> <u>SRV-028</u> </<u>점검항목</u>> <<u>점검항목명</u>> <u>원격 터미널 접속 타임아웃 설정 미흡</u> </<u>점검항목명</u>> <<u>점검 결과</u>>Y</<u>점검 결과</u>> <<u>점검 내용</u>> <u>TMOUT</u> 설정이 600 이하로 설정되어 있으므로 양호함 [현황] </<u>점검 내용</u>>

• LINUX SRV-028 “원격 터미널 접속 타임아웃 설정 미흡” 스크립트 수정 내역

항목 코드	스크립트 수정 전 (24년도)	스크립트 수정 후 (25년도)
LINUX SRV-028	1) <u>STIMEOUT 600</u> <u>미출력</u> <u>STIMEOUT</u> =`grep -i 'TMOUT' /etc/profile grep '\$([["\$tty)"	1) <u>STIMEOUT 600</u> 으로 출력 되도록 수정 <u>STIMEOUT</u> =`grep -i 'TMOUT' /etc/profile grep '600' grep -oE '[0-9]+' grep '^600\$'

• LINUX SRV-028 “원격 터미널 접속 타임아웃 설정 미흡” 스크립트 비교

Shell ▾

복사 캡션 ...

2024년도 Linux SRV-028 "원격 터미널 접속 타임아웃 설정 미흡" (Shellscript 개선 전)

```
session_timeout() {
    iCODE="SRV-028"
    iTITLE="원격 터미널 접속 타임아웃 설정 미흡"
    Item_head "$iCODE" "$iTITLE"

    if [ -f /etc/profile ]
    then
        if [ `grep -i "TMOUT" /etc/profile | grep -v "^#" | wc -l` -gt 0 ]
        then
            TIMEOUT=`grep -i 'TMOUT' /etc/profile | grep '$([[ "$tty)"`
            if [ $TIMEOUT -gt 600 ]
            then
                Result="N"
                Exp="TMOUT설정이 600이상으로 설정되어 있지 않으므로 취약함"
                Evi=`grep -i "TMOUT" /etc/profile | grep -v "^#"`
            else
                Result="Y"
                Exp="TMOUT설정이 600이상으로 설정되어 있으므로 양호함"
                Evi=`grep -i "TMOUT" /etc/profile | grep -v "^#"`
            fi
        else
            Result="N"
            Exp="TMOUT설정이 적용되어 있지 않으므로 취약함 / 양호기준 : TMOUT=600 (단위:초)"
            Evi=""
        fi
    fi

    Item_foot "$Result" "$Exp" "$Evi"
}
```

Shell ▾

복사 캡션 ...

2025년도 Linux SRV-028 "원격 터미널 접속 타임아웃 설정 미흡" (Shellscript 개선 후)

```
session_timeout() {
    iCODE="SRV-028"
    iTITLE="원격 터미널 접속 타임아웃 설정 미흡"
    Item_head "$iCODE" "$iTITLE"

    if [ -f /etc/profile ]
    then
        if [ `grep -i "TMOUT" /etc/profile | grep -v "^#" | wc -l` -gt 0 ]
        then
            TIMEOUT=`grep -i 'TMOUT' /etc/profile | grep '600' | grep -oE '[0-9]+' | gre
p '^600$`
            if [ $TIMEOUT -gt 600 ]
            then
                Result="N"
                Exp="TMOUT설정이 600이상으로 설정되어 있지 않으므로 취약함"
                Evi=`grep -i "TMOUT" /etc/profile | grep -v "^#"`
            else
                Result="Y"
                Exp="TMOUT설정이 600이상으로 설정되어 있으므로 양호함"
                Evi=`grep -i "TMOUT" /etc/profile | grep -v "^#"`
            fi
        else
            Result="N"
            Exp="TMOUT설정이 적용되어 있지 않으므로 취약함 / 양호기준 : TMOUT=600 (단위:초)"
            Evi=""
        fi
    fi

    Item_foot "$Result" "$Exp" "$Evi"
}
```

- LINUX SRV-042 “웹 서비스 상위 디렉터리 접근 제한 설정 미흡” ShellScript 개선 전후 스크립트 실행 결과

항목 코드	스크립트 실행 결과 (24년도)	스크립트 실행 결과 (25년도)
LINUX <u>SRV-042</u>	<<u>자산명</u>> <u>자산명</u> </<u>자산명</u>> <IP주소> 172.16.X.X </IP주소> <플랫폼> OS </플랫폼> <소프트웨어> 리눅스 </소프트웨어> <점검항목> <u>SRV-042</u> </점검항목> <점검항목명> <u>웹 서비스 상위 디렉터리 접근 제한 설정 미흡</u> </점검항목명> <점검 결과>Y</점검 결과> <점검 내용> [현황] </점검 내용>	<<u>자산명</u>> <u>자산명</u> </<u>자산명</u>> <IP주소> 172.16.X.X </IP주소> <플랫폼> OS </플랫폼> <소프트웨어> 리눅스 </소프트웨어> <점검항목> <u>SRV-042</u> </점검항목> <점검항목명> <u>웹 서비스 상위 디렉터리 접근 제한 설정 미흡</u> </점검항목명> <점검 결과>Y</점검 결과> <점검 내용> [현황] </점검 내용>

• LINUX SRV-042 “웹 서비스 상위 디렉터리 접근 제한 설정 미흡” 스크립트 수정 내역

항목 코드	스크립트 수정 전 (24년도)	스크립트 수정 후 (25년도)
LINUX SRV-042	1) 취약 버전 <u>미수정</u> 2) [현황]값에 최신 버전 정보 <u>미출력</u>	1) 취약 버전 수정 <u>vuln_ver</u> ="2.4.50" 2) 기존 <u>로직</u> 에서 최신 버전 여부 확인이 어려워 <u>current_ver</u> & <u>vuln_ver</u> 변수에 값(숫자)를 입력 받아 문자열로 치환하여 정상 동작하도록 수정 <u>current_ver</u> =\$(<u>\$APACHE_BIN_PATH</u> -V grep "Server version" <u>awk</u> '{print \$3}' cut -d'/' -f2)

• LINUX SRV-042 “웹 서비스 상위 디렉터리 접근 제한 설정 미흡” 스크립트

Shell ▾

복사 캡션 ...

2025년도 Linux SRV-042 “웹 서비스 상위 디렉터리 접근 제한 설정 미흡” (Shellscript 개선 후)

```
apache_allowoverride() {  
    if [ "$HTTP_SVR" = "true" ]; then  
        if [ -f "$httpd_temp" ]; then  
            if [ "$Proxy" = "true" ]; then  
                Result="Y"  
                Exp="Apache 서비스를 Proxy로 설정하여 사용하고 있으므로 양호함"  
            else  
                current_ver=$(($APACHE_BIN_PATH -V | grep "Server version" | awk '{print  
$3}' | cut -d'/' -f2)  
                # awk { print $3 } 명령어를 통해 APACHE_BIN_PATH의 내용 중 3열에 위치한 단어를 출력 후, cu  
t -d '/' f2 명령어로 3열에서 출력한 단어를 /를 기준으로 뒤에(2번째)있는 문자만 출력하여 cur  
rent_ver 변수에 입력  
                # Server version: Apahce/2.4.57 이란 문장에서 3번째 열은 Apache/2.4.57에 해당하고 /을 기준  
으로 뒤에 있는 문자는 "2.4.57" 이므로 "2.4.57"을 current_ver 변수에 입력  
                vuln_ver="2.4.50"  
                # 취약한 버전인 "2.4.50"을 vuln_ver 변수에 입력  
                if [ -n "$current_ver" ]; then  
                    if [ "$(printf '%s\n%s\n' "$current_ver" "$vuln_ver" | sort -V | hea  
d -n1)" = "$current_ver" ]; then  
                        allow_override=$(grep -v "^ #" "$httpd_temp" | grep -B5 "AllowOv  
erride" | grep -i "AllowOverride")  
                        # current_ver 변수가 비어있지 않다면 날개 안 명령어 실행. %s\n 2번 실행 통해 current_ver  
와 vuln_ver의 값을 개행으로 나눠 정렬한 후 sort -V로 버전 오름차순으로 정렬한후 head -n1로  
1행(current_ver과 vuln_ver 변수에 입력된 값 중 숫자가 낮은 버전)이 $current_ver과 일치한다  
면, $httpd_temp 파일의 스크립트를 "^ #" 등 불필요한 값을 제거한 후, AllowOverride 키워드  
가 포함된 행을 포함 그 이전 5개 행까지 allow_override 변수에 입력  
                        if [ -z "$allow_override" ]; then  
                            Result="N"  
                            Exp="Apache 실행 중이며 취약점 기준 버전 이하를 사용 중이고 Al  
lowOverride 설정이 없어 취약함"  
                        # current_ver 변수가 비어있다면 [점검 결과]를 "취약/N", [점검 내용]의 [현황]에 "Apache 실행  
중이며 취약점 기준 버전 이하를 사용 중이고 AllowOverride 설정이 없어 취약함"를 출력  
                        fi  
                    fi  
                fi  
            fi  
        fi  
    fi  
}
```

• LINUX SRV-108 “로그에 대한 접근통제 및 관리 미흡” ShellScript 개선 전후 스크립트 실행 결과

항목 코드	스크립트 실행 결과 (24년도)	스크립트 실행 결과 (25년도)
	<<u>자산명</u>> <u>자산명</u> </<u>자산명</u>> <<u>IP주소</u>> 172.16.X.X </<u>IP주소</u>> <<u>플랫폼</u>> OS </<u>플랫폼</u>> <<u>소프트웨어</u>> 리눅스 </<u>소프트웨어</u>> <<u>점검항목</u>> <u>SRV-108</u> </<u>점검항목</u>> <<u>점검항목명</u>> 로그에 대한 접근통제 및 관리 미흡 </<u>점검항목명</u>> <점검 결과>Y</점검 결과>	<<u>자산명</u>> <u>자산명</u> </<u>자산명</u>> <<u>IP주소</u>> 172.16.X.X </<u>IP주소</u>> <<u>플랫폼</u>> OS </<u>플랫폼</u>> <<u>소프트웨어</u>> 리눅스 </<u>소프트웨어</u>> <<u>점검항목</u>> <u>SRV-108</u> </<u>점검항목</u>> <<u>점검항목명</u>> 로그에 대한 접근통제 및 관리 미흡 </<u>점검항목명</u>> <점검 결과>Y</점검 결과>
LINUX <u>SRV-108</u>	<점검 내용> 시스템 <u>로그파일</u> 권한을 644이하로 설정하고 있으므로 양호함 [현황] -rw-r----- 1 root root 0 Jan 1 00:00 /var/log/<u>cron</u> </점검 내용>	<점검 내용> LOG 디렉터리 내 <u>로그파일</u> 권한 644 이하로 설정되어 있으므로 양호함 (<u>btmtp</u>, <u>wtmp</u>, <u>lastlog</u> <u>로그파일</u>은 수정 불가이므로 각각 660, 644로 기본 설정) [현황] -rw-r----- 1 root root 0 Jan 1 00:00 /var/log/<u>cron</u> -rw-rw-r-- 1 root <u>utmp</u> 0 Jan 1 00:00 /var/log/<u>btmtp</u> -rw-rw-r-- 1 root <u>utmp</u> 0 Jan 1 00:00 /var/log/<u>lastlog</u> -rw-rw-r-- 1 root <u>utmp</u> 0 Jan 1 00:00 /var/log/<u>wtmp</u> -rw-r----- 1 root root 0 Jan 1 00:00 /dev/<u>console</u> </점검 내용>

• LINUX SRV-108 “로그에 대한 접근통제 및 관리 미흡” 스크립트 수정 내역

항목 코드	스크립트 수정 전 (24년도)	스크립트 수정 후 (25년도)
<u>LINUX SRV-108</u>	1) 현황에 권한 600(<u>rw.....</u>) <u>로그파일 미출력</u> <u>S_PERM2</u>="<u>^</u>-<u>rw.r.....*</u> <u>^</u>-<u>rw.r..r..*</u> <u>^</u>-<u>rw.rw.r..*</u> <u>^</u>-<u>rw.rw....*</u> " 2) <u>wtmp/btmp/lastlog</u> <u>미출력</u>	1) 현황에 권한 600(<u>rw.....</u>) <u>로그파일 출력</u> <u>S_PERM2</u>="<u>^</u>-<u>rw.r.....*</u> <u>^</u>-<u>rw.r..r..*</u> <u>^</u>-<u>rw.rw.r..*</u> <u>^</u>-<u>rw.rw....*</u> <u>^</u>-<u>rw.....*</u>" 2) <u>wtmp/btmp/lastlog</u> <u>출력</u> <u>Evi</u>='<u>ls -al \$DIR_SYSLOG_FILE grep -E \$S_PERM2;echo "\n[참고]log 디렉터리 전체 파일\n";ls -al \$DIR_SYSLOG_FILE`</u>

• LINUX SRV-108 “로그에 대한 접근통제 및 관리 미흡” 스크립트 비교

Shell ▾

복사 캡션 ...

2024년 도 Linux SRV-108 "로그에 대한 접근통제 및 관리 미흡" (Shellscript 개선 전)

```
systemlogfile_perm() {
    iCODE="SRV-108"
    iTitle="로그에 대한 접근통제 및 관리 미흡"
    Item_head "$iCODE" "$iTitle"

    DIR_SYSLOG_FILE="/var/log/messages* /dev/console* /var/log/secure* /var/log/maillog*
/var/log/boot*.log /var/log/dmesg* /var/log/cron* /var/log/xferlog* /var/log/httpd/access
*_log /var/log/error*_log"

    if [ `ls -al $DIR_SYSLOG_FILE | grep -E "^-rwx.....*|^-rw..w.....*|-rw...x.....*|^-r
w.....w.....*|^-rw.....x.*" wc -l` -gt 0 ]
    then
        Result="N"
        Exp="시스템 로그파일 권한을 644이상으로 설정하여 취약함"
        Evi=`ls -al $DIR_SYSLOG_FILE | grep -E "^-rwx.....*|^-rw..w.....*|-rw...x.....*|^-r
w.....w.....*|^-rw.....x.*" wc -l`
        echo -e "\n [참고]\n" `ls -al $DIR_SYSLOG_FILE`
    else
        Result="Y"
        Exp="시스템 로그파일 권한을 644이하로 설정하고 있어 양호함"
        Evi=`ls -al $DIR_SYSLOG_FILE`
    fi

    test=`ls -al $DIR_SYSLOG_FILE | wc -l`

    if [ $test -eq 0 ]
    then
        Result="M"
        Exp="시스템 로그파일이 존재하지 않아 확인 필요"
        Evi=`ls -al /var/log`
    fi

    Item_foot "$Result" "$Exp" "$Evi"

    unset test
}
```

Shell ▾

복사 캡션 ...

2025년 도 Linux SRV-108 "로그에 대한 접근통제 및 관리 미흡" (Shellscript 개선 후)

```
systemlogfile_perm() {
    iCODE="SRV-108"
    iTitle="로그에 대한 접근통제 및 관리 미흡"
    Item_head "$iCODE" "$iTitle"

    DIR_SYSLOG_FILE="/var/log/btmp* /var/log/wtmp* /var/log/lastlog* /var/log/message* /d
ev/console* /var/log/secure* /var/log/maillog* /var/log/boot*.log /var/log/dmesg* /var/lo
g/cron* /var/log/xferlog* /var/log/httpd/access*_log /var/log/error*_log*"

    # 검사할 로그 파일 경로를 DIR_SYSLOG_FILE 변수에 선언
    V_PERM1="^-rw.rw.rw.*|^-rw.rw.rwx*|^-rw.rwx.rw.*|^-rwxrwxrwx*"
    # 취약한 권한을 V_PERM1 변수에 선언 (666, 667, 777)
    S_PERM2="^-rw.r.....*|^-rw.r..r..*|^-rw.rw.r..*|^-rw.rw....*|^-rw.....*"
    # 양호한 권한을 S_PERM2 변수에 선언 (640, 644, 664, 660, 600)

    if [ `ls -al $DIR_SYSLOG_FILE | wc -l` -eq 0 ]
    then
        Result="M"
        Exp="시스템 로그파일이 존재하지 않아 확인필요"
        Evi=`ls -al /var/log`
        # DIR_SYSLOG_FILE이 비어있으면 인터뷰/확인필요
    else
        if [ `ls -al $DIR_SYSLOG_FILE | grep -E $V_PERM1 | wc -l` -gt 0 ]
        then
            Result="N"
            Exp="log 디렉터리 내 로그파일 권한은 644이하로 설정하지 않으므로 취약함(btmp,
wtmp, lastlog 로그파일은 수정할 수 없으므로 각각 660, 664로 설정되어 있음)"
            Evi=`ls -al $DIR_SYSLOG_FILE | grep -E $V_PERM1;echo "\n[참고]log 디렉터리 전
체 파일\n";ls -al $DIR_SYSLOG_FILE`
        fi
        # DIR_SYSLOG_FILE의 V_PERM1이 비어있으면 양호
    fi

    Item_foot "$Result" "$Exp" "$Evi"
}
```

• LINUX SRV-127 “계정 잠금 임계값 설정 미비” ShellScript 개선 전후 스크립트 실행 결과

항목 코드	스크립트 실행 결과 (24년도)	스크립트 실행 결과 (25년도)
LINUX SRV-127	<자산명> 자산명 </자산명> <IP주소> 172.16.X.X </IP주소> <플랫폼> OS </플랫폼> <소프트웨어> 리눅스 </소프트웨어> <점검항목> SRV-108 </점검항목> <점검항목명> 계정 잠금 임계값 설정 미비 </점검항목명> <점검 결과> Y </점검 결과> <점검 내용> 계정 잠금 임계값이 5 이하로 설정되어 있으므로 양호함 [현황] deny = 5 </점검 내용>	<자산명> 자산명 </자산명> <IP주소> 172.16.X.X </IP주소> <플랫폼> OS </플랫폼> <소프트웨어> 리눅스 </소프트웨어> <점검항목> SRV-108 </점검항목> <점검항목명> 계정 잠금 임계값 설정 미비 </점검항목명> <점검 결과> Y </점검 결과> <점검 내용> (FILE PATH: /etc/security/faillock.conf) 계정 잠금 임계값이 5 이하로 설정되어 있으므로 양호함 [현황] deny = 5 </점검 내용>

• LINUX SRV-127 “계정 잠금 임계값 설정 미비” 스크립트 수정 내역

항목 코드	스크립트 수정 전 (24년도)	스크립트 수정 후 (25년도)
LINUX SRV-127	1) 기존 로직에 <u>pam.d</u> 설정 파일 경로 <u>미설정</u> <pre>if [`echo \$<u>pam_tally</u> 경로`] then <u>결과값</u> : "Y" <u>점검 내용</u> : "잠금 <u>임계값</u> 관련 설정이 없으나, 하단 결과 참고 후 진단 필요"</pre> 2) <u>tally2</u> 없이 <u>faillock</u>만 존재 <pre>if [-f /etc/<u>pam.d</u>/<u>password-auth</u>] then Result="Y" Exp="계정 잠금 <u>임계값</u>이 5 이하의 값으로 설정되어 있으므로 양호함"</pre>	1) 기존 로직에 <u>pam.d</u> 설정 파일 경로 <u>설정</u> <pre>if [`echo \$<u>pam_tally</u> 경로`] then <u>결과값</u> : "Y" <u>점검 내용</u> = "(파일 경로 : /etc/<u>pam.d</u>/<u>common-auth</u>) 잠금 <u>임계값</u> 관련 설정이 없으나, 하단 결과 참고 후 진단 필요"</pre> 2) <u>tally2</u> 추가 <u>추가 이유</u> : Redhat 6.x 이하 레거시 등에 <u>faillock</u>이 도입되지 않으므로 추가 <pre>if [-f /etc/<u>pam.d</u>/<u>password-auth</u>] then <u>deny_value1</u>=`grep "auth" /etc/<u>pam.d</u>/<u>password-auth</u>   grep "<u>pam faillock</u>"   grep "deny"   grep -v "^#"   <u>awk</u> '{for(i=1;i<=NF;i++) if(index(\$i, "deny")) print \$ i }` <u>deny_value2</u>=`grep -E 'tally\.so <u>tally2</u>\.so' /etc/<u>pam.d</u>/<u>system-auth</u>   grep -oP 'deny=\K[0-9]+'`</pre> <u>코드 해석</u> : <u>deny_value1</u> 변수는 <u>password-auth</u>의 <u>faillock</u>, <u>deny_value2</u> 변수는 <u>password-auth</u>의 <u>tally2</u> 존재 여부를 비교 후 점검 결과(양호/취약)를 얻음

• LINUX SRV-127 “계정 잠금 임계값 설정 미비” 스크립트 비교

Shell

2024년도 Linux SRV-127 "계정 잠금 임계값 설정 미비" (Shellscript 개선 전)

account_lock() {
 iCODE="SRV-127"
 iTitle="계정 잠금 임계값 설정 미비"
 Item_head "\$iCODE" "\$iTitle"

 if [-f /etc/security/faillock.conf]; then
 deny_value=\$(grep -E "^s*deny\s*=" /etc/security/faillock.conf | grep -v "^s*
" | awk -F= '{print \$2}' | tr -d '[:space:]')
 if [-z "\$deny_value"] || ["\$deny_value" -gt 5]; then
 Result="N"
 Exp="계정 잠금 임계값이 설정되어 있지 않거나 5를 초과하여 취약함" ### EXP="(FI
LE PATH : /etc/security/faillock.conf) 계정 잠금 임계값이 설정되어 있지 않거나 5를 초과하
므로 취약함" 변경
 Evi=\$(grep -E "^s*deny\s*=" /etc/security/faillock.conf | grep -v "^s*#")
 else
 Result="Y"
 Exp="계정 잠금 임계값이 5 이하로 설정되어 있어 양호함" ### EXP="(FILE PATH : /
etc/security/faillock.conf) 계정 잠금 임계값이 5 이하로 설정되어 있어 양호함" 변경
 Evi=\$(grep -E "^s*deny\s*=" /etc/security//faillock.conf | grep -v "^s*#")
 fi

#Redhat 계열 리눅스의 인증 설정 파일
elif [-f /etc/pam.d/system-auth]
then
 # 잠금 임계값(deny) 관련 설정이 존재하는지 판별
 deny_value=`grep "auth" /etc/pam.d/system-auth | grep "pam_faillock" | grep "den
y" | grep -v "^#" | awk '{for(i=1;i<=NF;i++) if(index(\$i, "deny")) print \$i }`'
 ### deny_value2=`grep -E 'tally\.so|tally2\.so' /etc/pam.d/system-auth | grep -o
P 'deny=K[0-9]+'` 추가
 if [`echo \$deny\_value | grep -c '\n'` -eq 0]
 then
 Result1="Bad"
 else
 if [`expr "\$deny\_value" | cut -d"=" -f2` -gt 5]
 then
 Result1="Bad"
 else
 Result2="Good"
 fi
 fi

fi
#RHEL 6 이상(xwindow, ssh 관련)
if [-f /etc/pam.d/password-auth]
then
 #system-auth, password-auth 모두 존재하고, 두 파일 모두 계정 잠금 임계값
설정이 되어 있는 경우
 Result="Y"
 Exp="계정 잠금 임계값이 5 이하의 값으로 설정되어 있으므로 양호함"
 Evi="<system-auth\n`grep "auth" /etc/pam.d/system-auth | grep "pam_faill
ock```\n<password-auth>\n`grep "auth" /etc/pam.d/password-auth | grep "pam_faillock```\n
elif ["\$Result1" = "Good"] && ["\$Result2" = "N/A"]
 then
 #password-auth는 존재하지 않고, system-auth의 설정이 적절한 경우
 Result="Y"
 Exp="계정 잠금 임계값이 5 이하의 값으로 설정되어 있으므로 양호함"
 Evi="<system-auth>\n`grep "auth" /etc/pam.d/system-auth | grep "pam_fail
lock```\n<password-auth>\nNoFileExists"
 else
 #system-auth, password-auth 중 한 파일 또는 두 파일 모두 설정이 미흡한 경
우
 Result="N"
 Exp="계정 잠금 임계값이 존재하지 않거나, 5 초과와 값으로 설정되어 있으므로
취약함"
 Evi="<system-auth\n```\n`grep "auth" /etc/pam.d/system-auth | grep "pam_faill
ock```\n<password-auth>\n```\n`grep "auth" /etc/pam.d/password-auth | grep ""pam_faillock```\n
 fi
 #Devian 계열 리눅스의 인증 설정파일
elif [-f /etc/pam.d/comm-auth]
 then
 #잠금 임계값(deny) 관련 설정이 존재하는지 판별
 deny_value=`grep "auth" /etc/pam.d/common-auth | grep "pam_tally" | grep "den
y" | grep -v "^#" | awk '{for(i=1;i<=NF;i++) if(index(\$i, "deny")) print \$i }`'
 if [`echo \$deny\_value | grep -c '\n'` -eq 0]
 then
 Result="Y"
 Exp="잠금 임계값 관련 설정이 없으나, 하단 결과 참고 후 진단 필요"
 Evi=`grep "auth" /etc/pam.d/common-auth`
 else
 if [`expr "\$deny\_value" | cut -d"=" -f2` -gt 5]
 then
 Result="N"
 Exp="계정 잠금 임계값이 5 초과와 값으로 설정되어 있으므로 취약
함"
 Evi=`grep "auth" /etc/pam.d/common-auth | grep "pam_tally```
 else
 Result="Y"
 Exp="계정 잠금 임계값이 5 이하의 값으로 설정되어 있으므로 양호
함"
 Evi=`grep "auth" /etc/pam.d/common-auth | grep "pam_tally```
 fi
 fi
 else
 Result="M"
 Exp="계정 잠금 임계값 설정 파일이 존재하지 않으므로 수동진단 필요"
 Evi=""
 fi

 Item_foot "\$Result" "\$Exp" "\$Evi"
}

Shell

2025년 도 Linux SRV-127 "계정 잠금 임계값 설정 미비" (Shellscript 개선 후)

account_lock() {
 iCODE="SRV-127"
 iTitle="계정 잠금 임계값 설정 미비"
 Item_head "\$iCODE" "\$iTitle"

 if [-f /etc/security/faillock.conf]; then
 deny_value=\$(grep -E "^s*deny\s*=" /etc/security/faillock.conf | grep -v "^s*
" | awk -F= '{print \$2}' | tr -d '[:space:]')
 if [-z "\$deny_value"] || ["\$deny_value" -gt 5]; then
 Result="N"
 Exp="(FILE PATH : /etc/security/faillock.conf) 계정 잠금 임계값이 설정되어 있
지 않거나 5를 초과하므로 취약함" ### Exp="계정 잠금 임계값이 설정되어 있지 않거나 5를 초과
하여 취약함" 변경
 Evi=\$(grep -E "^s*deny\s*=" /etc/security/faillock.conf | grep -v "^s*#")
 else
 Result="Y"
 Exp="(FILE PATH : /etc/security/faillock.conf) 계정 잠금 임계값이 5 이하로 설
정되어 있어 양호함" ### 2024 Exp="계정 잠금 임계값이 5 이하로 설정되어 있어 양호함" 변경
 Evi=\$(grep -E "^s*deny\s*=" /etc/security//faillock.conf | grep -v "^s*#")
 fi

#Redhat 계열 리눅스의 인증 설정 파일
elif [-f /etc/pam.d/system-auth]
then
 # 잠금 임계값(deny) 관련 설정이 존재하는지 판별
 deny_value=`grep "auth" /etc/pam.d/system-auth | grep "pam_faillock" | grep "den
y" | grep -v "^#" | awk '{for(i=1;i<=NF;i++) if(index(\$i, "deny")) print \$i }`'
 deny_value2=`grep -E 'tally\.so|tally2\.so' /etc/pam.d/system-auth | grep -oP 'de
ny=K[0-9]+'` ### 2025 추가
 if [`deny\_value1 | wc -l` -eq 0]
 then
 Result1="Bad"
 else
 if [\$deny_value2 -gt 5]
 then
 Result1="Bad"
 else
 Result2="Good"
 fi
 fi

fi
#RHEL 6 이상(xwindow, ssh 관련)
if [-f /etc/pam.d/password-auth]
then
 #잠금 임계값(deny) 관련 설정이 존재하는지 판별
 deny_value1=`grep "auth" /etc/pam.d/password-auth | grep "pam_faillock" | gre
p "deny" | grep -v "^#" | awk '{for(i=1;i<=NF;i++) if(index(\$i, "deny")) print \$i }`'
 deny_value2=`grep -E 'tally\.so|tally2\.so' /etc/pam.d/system-auth | grep -o
P 'deny=K[0-9]+'`
 if [`deny\_value1 | wc -l` -eq 0]
 then
 if [`deny\_value2 | wc -l` -eq 0]
 then
 Result2="Bad"
 else
 if [\$deny_value2 -gt 5]
 then
 Result2="Bad"
 else
 Result2="Good"
 fi
 fi
 else
 if [\$deny_value1 -gt 5]
 then
 Result2="Bad"
 else
 Result2="Good"
 fi
 fi
 else
 Result2="N/A"
 fi
 #system-auth, password-auth 모두 설정이 되어 있어야 양호로 판단함
 if [\$Result1 = "Good"] && [\$Result2 = "Good"]
 then
 #system-auth, password-auth 모두 존재하고, 두 파일 모두 계정 잠금 임계값 설정
이 되어 있는 경우
 Result="Y"
 Exp="(FILE PATH : /etc/pam.d/system-auth) 계정 잠금 임계값이 5 이하의 값으로
설정되어 있으므로 양호함" ### 2025 파일 경로 추가
 Evi="<system-auth>\n```\n`grep "auth" /etc/pam.d/system-auth | grep -E "pam_faill
ock|pam_tally|pam_tally2```\n<password-auth>\n```\n`grep "auth" /etc/pam.d/password-auth | g
rep -E "pam_faillock|pam_tally|pam_tally2```\n
 elif [\$Result1 = "Good"] && [\$Result2 = "N/A"]
 then
 #password-auth는 존재하지 않고, system-auth의 설정이 적절한 경우
 Result="Y"
 Exp="(FILE PATH : /etc/pam.d/system-auth) 계정 잠금 임계값이 5 이하의 값으로
설정되어 있으므로 양호함"
 Evi="<system-auth>\n```\n`grep "auth" /etc/pam.d/system-auth | grep -E "pam_faill
ock|pam_tally|pam_tally2```\n<password-auth>\nNoFileExists"
 else
 #system-auth, password-auth 중 한 파일 또는 두 파일 모두 설정이 미흡한 경우
 Result="N"
 Exp="(FILE PATH : /etc/pam.d/system-auth) 계정 잠금 임계값이 존재하지 않거나,
5 초과와 값으로 설정되어 있으므로 취약함"
 Evi="<system-auth>\n```\n`grep "auth" /etc/pam.d/system-auth | grep -E "pam_faill
ock|pam_tally|pam_tally2```\n<password-auth>\n```\n`grep "auth" /etc/pam.d/password-auth | gr
ep -E "pam_faillock|pam_tally|pam_tally2```\n
 fi
 #Devian 계열 리눅스의 인증 설정파일
 elif [-f /etc/pam.d/comm-auth]
 then
 #잠금 임계값(deny) 관련 설정이 존재하는지 판별
 deny_value=`grep "auth" /etc/pam.d/common-auth | grep "pam_tally" | grep "den
y" | grep -v "^#" | awk '{for(i=1;i<=NF;i++) if(index(\$i, "deny")) print \$i }`'
 if [`echo \$deny\_value | grep -c '\n'` -eq 0]
 then
 Result="Y"
 Exp="(FILE PATH : /etc/pam.d/common-auth) 잠금 임계값 관련 설정이 없
으나, 하단 결과 참고 후 진단 필요" ### 2025 FILE PATH 추가
 Evi=`grep "auth" /etc/pam.d/common-auth`
 else
 if [`expr "\$deny\_value" | cut -d"=" -f2` -gt 5]
 then
 Result="N"
 Exp="(FILE PATH : /etc/pam.d/common-auth) 계정 잠금 임계값이
5 초과와 값으로 설정되어 있으므로 취약함" ### 2025 FILE PATH 추가
 Evi=`grep "auth" /etc/pam.d/common-auth | grep "pam_tally```
 else
 Result="Y"
 Exp="(FILE PATH : /etc/pam.d/common-auth) 계정 잠금 임계값이
5 이하의 값으로 설정되어 있으므로 양호함" ### 2025 FILE PATH 추가
 Evi=`grep "auth" /etc/pam.d/common-auth | grep "pam_tally```
 fi
 fi
 else
 Result="M"
 Exp="계정 잠금 임계값 설정 파일이 존재하지 않으므로 수동진단 필요"
 Evi=""
 fi
 else
 Result="M"
 Exp="계정 잠금 임계값 설정 파일이 존재하지 않으므로 수동진단 필요"
 Evi=""
 fi

 Item_foot "\$Result" "\$Exp" "\$Evi"
}

• LINUX SRV-148 “웹 서비스 정보 노출” ShellScript 개선 전후 스크립트 실행 결과

항목 코드	스크립트 실행 결과 (24년도)	스크립트 실행 결과 (25년도)
LINUX <u>SRV-148</u> (양호)	<<u>자산명</u>> <u>자산명</u> </<u>자산명</u>> <<u>IP주소</u>> 172.16.X.X </<u>IP주소</u>> <<u>플랫폼</u>> OS </<u>플랫폼</u>> <<u>소프트웨어</u>> 리눅스 </<u>소프트웨어</u>> <<u>점검항목</u>> <u>SRV-148</u> </<u>점검항목</u>> <<u>점검항목명</u>> 웹 서비스 정보 노출 </<u>점검항목명</u>> <<u>점검 결과</u>> Y </<u>점검 결과</u>> <점검 내용> Apache 사용하지 않으므로 양호함 [현황] <u>현황값 없음</u> </점검 내용>	<<u>자산명</u>> <u>자산명</u> </<u>자산명</u>> <<u>IP주소</u>> 172.16.X.X </<u>IP주소</u>> <<u>플랫폼</u>> OS </<u>플랫폼</u>> <<u>소프트웨어</u>> 리눅스 </<u>소프트웨어</u>> <<u>점검항목</u>> <u>SRV-148</u> </<u>점검항목</u>> <<u>점검항목명</u>> 웹 서비스 정보 노출 </<u>점검항목명</u>> <<u>점검 결과</u>> Y </<u>점검 결과</u>> <점검 내용> Connector 태그의 SERVER 옵션이 설정되어 있으므로 양호함 (Tomcat 경로 : <u>/sw/tomcat/xx/xx/server.xml</u>) [현황] <Connector port="8080" protocol="HTTP/1.1" <u>connectionTimeout="20000"</u> <u>redirectPort="8443"</u> <u>maxParameterCount="1000"</u> server= "<u>APPM for Password Web Client</u>" > </점검 내용>
LINUX <u>SRV-148</u> (취약)	<<u>자산명</u>> <u>자산명</u> </<u>자산명</u>> <<u>IP주소</u>> 172.16.X.X </<u>IP주소</u>> <<u>플랫폼</u>> OS </<u>플랫폼</u>> <<u>소프트웨어</u>> 리눅스 </<u>소프트웨어</u>> <<u>점검항목</u>> <u>SRV-148</u> </<u>점검항목</u>> <<u>점검항목명</u>> 웹 서비스 정보 노출 </<u>점검항목명</u>> <<u>점검 결과</u>> N </<u>점검 결과</u>> <점검 내용> <u>ServerTokens</u> 설정값이 없으므로 취약함 [현황] Apache 프로세스 리스트 : root 00000 0 0 00:00 00:00:00 ./httpd start daemon 00000 0 0 00:00 00:00:00 ./httpd start </점검 내용>	<<u>자산명</u>> <u>자산명</u> </<u>자산명</u>> <<u>IP주소</u>> 172.16.X.X </<u>IP주소</u>> <<u>플랫폼</u>> OS </<u>플랫폼</u>> <<u>소프트웨어</u>> 리눅스 </<u>소프트웨어</u>> <<u>점검항목</u>> <u>SRV-148</u> </<u>점검항목</u>> <<u>점검항목명</u>> 웹 서비스 정보 노출 </<u>점검항목명</u>> <<u>점검 결과</u>> N </<u>점검 결과</u>> <점검 내용> <u>ServerTokens</u> 설정값이 없으므로 취약함 (Apache Conf 경로 : <u>/sw/apache/xx/xx/httpd.conf</u>) [현황] Default <u>ServerTokens</u> 설정 현황 : # <u>ServerTokens</u> <u>ServerTokens Full</u> </점검 내용>

• LINUX SRV-148 “웹 서비스 정보 노출” 스크립트 수정 내역

항목 코드	스크립트 수정 전 (24년도)	스크립트 수정 후 (25년도)
LINUX SRV-148	1) [점검 내용]에 설정 파일 경로 미출력 if [`cat \$httpd_temp   grep -v "^ *#"   grep "ServerTokens"   wc -l` -eq 0] then Result="N" Exp="<u>ServerTokens</u> 설정값이 없으므로 취약함" 2) [현황]에 <u>ServerSignature</u> 사용 여부 미출력 if [`cat \$httpd_temp   grep -v "^ *#"   grep "ServerTokens *Prod"   wc -l` -eq `cat \$httpd_temp   grep -v "^ *#"   grep "ServerTokens"   wc -l`] then Result="Y" Exp="<u>ServerTokens</u> 지시자의 옵션이 Prod 설정이 되어 있으므로 양호함" else Result="N" Exp="<u>ServerTokens</u> 지시자의 옵션이 Prod가 아니므로 취약함" 3) Tomcat 서비스 미출력 -	1) [점검 내용]에 설정 파일 경로 출력 if [`cat \$httpd_temp   grep -v "^ *#"   grep "ServerTokens"   wc -l` -eq 0] then Result="N" Exp="<u>ServerTokens</u> 설정값이 없으므로 취약함 (Apache Conf Path : \$httpd_temp)" 2) [현황]에 <u>ServerSignature</u> 사용 여부 출력 if [`cat \$httpd_temp   grep -v "^ *#"   grep "ServerTokens *Prod"   wc -l` -eq 1] then if [`cat \$httpd_temp   grep -v "^ *#"   grep "<u>ServerSignature</u> *Off"   wc -l` -eq 1] then Result="Y" Exp="<u>ServerTokens</u> 지시자의 옵션이 Prod 및 <u>ServerSignature</u> 설정이 Off 설정되어 있으므로 양호함 (Apache Conf Path : \$httpd_temp)" 3) Tomcat 서비스 추가 및 출력 if [-f \$TOMCAT_SERVERXML_FILE] then if [`awk '/<Connector /,/>/' \$TOMCAT_SERVERXML_FILE   grep -i server   sed 's/[ \t]//g'   wc -l` -eq 0] then Result="M" Exp="<u>Connector</u> 태그의 SERVER 옵션이 미설정 시 헤더에 서버 정보가 노출되나, 별도 보안 설정 확인 (Tomcat SERVER.XML Path : \$TOMCAT_SERVERXML_FILE)" else Result="Y" Exp="<u>Connector</u> 태그의 SERVER 옵션이 설정되어 있으므로 양호함 (Tomcat SERVER.XML Path : \$TOMCAT_SERVER_FILE)"



2024년도 Linux SRV-148 "웹 서비스 정보 노출" (Shellscript 개선 전)

```
apache_servertokens() {
    iCODE="SRV-148"
    iTitle="웹 서비스 정보 노출"
    Item_head "$iCODE" "$iTITLE"

    if [ $HTTP_SVR = "true" ]
    then
        if [ -f $httpd_temp ]
        then
            if [ $Proxy = "true" ]
            then
                Result="Y"
                Exp="Apache 서비스를 Proxy 설정하여 사용하고 있으므로 양호함" ## 25년도 추가 내용 : (Apache Conf Path : $httpd_temp)"
                Evi=`echo $ProxyTest;echo "\n\n설정확인";cat $httpd_temp | grep "ServerTokens";echo "\napache 프로세스 확인 ";ps -ef | grep -i "httpd" | grep -v "grep"`
            else
                if [ `cat $httpd_temp | grep -v "^ *#" | grep "ServerTokens" | wc -l` -eq 0 ]
                then
                    Result="N"
                    Exp="ServerTokens 설정값이 없으므로 취약함" ## 25년도 추가 내용 : (Apache Conf Path : $httpd_temp)"
                    Evi=`echo "설정확인";cat $httpd_temp | grep "ServerTokens";echo "\napache 프로세스 확인 ";ps -ef | grep -i "httpd" | grep -v "grep"`
                    # 25년도 `cat $httpd_temp | grep "ServerTokens";echo "\nDefault ServerTokens 설정현황 :";cat $httpd_Default_temp | grep -i "ServerTokens"
                else
                    if [ `cat $httpd_temp | grep -v "^ *#" | grep "ServerTokens *Prod" | wc -l` -eq `cat $httpd_temp | grep -v "^ *#" | grep "ServerTokens" | wc -l` ]
                    then
                        Result="Y"
                        Exp="ServerTokens 지시자의 옵션이 Prod 설정이 되어 있으므로 양호함"
                    else
                        Result="N"
                        Exp="ServerTokens 지시자의 옵션이 Prod가 아니므로 취약함"
                    fi
                    Evi=`echo "설정확인";cat $httpd_temp | grep "ServerTokens";echo "\napache 프로세스 확인 ";ps -ef | grep -i "httpd" | grep -v "grep"`
                fi
            fi
        else
            Result="Y"
            Exp="Apache 서비스 사용하지 않으므로 해당사항 없음"
            Evi=""
        fi

        Item_foot "$Result" "$Exp" "$Evi"
    }
}
```

Shell

복사 캡션 ...

2025년도 Linux SRV-148 "웹 서비스 정보 노출" (Shellscript 개선 후)

```
apache_servertokens() {
    iCODE="SRV-148"
    iTitle="웹 서비스 정보 노출"
    Item_head "$iCODE" "$iTITLE"

    if [ $HTTP_SVR = "true" ]
    then
        if [ -f $httpd_temp ]
        then
            if [ $Proxy = "true" ]
            then
                Result="Y"
                Exp="Apache 서비스를 Proxy 설정하여 사용하고 있으므로 양호함 (Apache Conf Path : $httpd_temp)"
                Evi=`echo $ProxyTest;echo "\n\n설정확인";cat $httpd_temp | grep "ServerTokens";echo "\napache 프로세스 확인 ";ps -ef | grep -i "httpd" | grep -v "grep"`
            else
                if [ `cat $httpd_temp | grep -v "^ *#" | grep "ServerTokens" | wc -l` -eq 0 ]
                then
                    Result="N"
                    Exp="ServerTokens 설정값이 없으므로 취약함 (Apache Conf Path : $httpd_temp)"
                    Evi=`cat $httpd_temp | grep "ServerTokens";echo "\nDefault ServerTokens 설정현황 :";cat $httpd_Default_temp | grep -i "ServerTokens"
                    else
                        if [ `cat $httpd_temp | grep -v "^ *#" | grep "ServerTokens *Prod" | wc -l` -eq 1 ]
                        then
                            if [ `cat $httpd_temp | grep -v "^ *#" | grep "ServerSignature *Off" | wc -l` -eq 1 ]
                            then
                                Result="Y"
                                Exp="ServerTokens 지시자의 옵션이 Prod 및 ServerSignature 설정이 Off 설정되어 있으므로 양호함 (Apache Conf Path : $httpd_temp)"
                                Evi=`echo "ServerTokens 지시자 옵션\n";cat $httpd_temp | grep "ServerTokens";echo "\n";echo "\nServerSignature 설정현황 :";cat $httpd_temp | grep "ServerSignature"`
                            else
                                Result="N"
                                Exp="ServerTokens 지시자의 옵션이 Prod로 설정되어 있으나, ServerSignature 설정 확인 필요 (Apache Conf Path : $httpd_temp)"
                                Evi=`echo "ServerTokens 지시자 옵션\n";cat $httpd_temp | grep "ServerTokens";echo "\nServerSignature 설정현황 : ";cat $httpd_Default_temp | grep "ServerSignature"`
                            fi
                        else
                            Result="N"
                            Exp="ServerTokens 지시자의 옵션이 Prod가 아니므로 취약함 (Apache Default Conf Path : $APACHE_ROOT/conf/httpd-default.conf)"
                            Evi=`cat $httpd_Default_temp | grep -i "ServerTokens";echo "\nDefault ServerSignature 설정현황 :";cat $httpd_Default_temp | grep -i "ServerSignature"`
                            fi
                    fi
                fi
            fi
        else
            Result="M"
            Exp="Apache 서비스 실행하고 있으나, httpd.conf 파일을 찾을 수 없으므로 수동 점검"
            Evi="$APACHE_ROOT의 파일\n`ls -alR $APACHE_ROOT;ps -ef | grep -i "httpd" | grep -v "grep"`
        fi
    elif [ $HTTPWAS_SVR = "true" ]
    then
        if [ -f $TOMCAT_SERVERXML_FILE ]
        then
            then
                if [ `awk '/<Connector /,/>/' $TOMCAT_SERVERXML_FILE | grep -i server | sed 's/[ \t]//g' | wc -l` -eq 0 ]
                then
                    Result="M"
                    Exp="Connector 태그의 SERVER 옵션이 미설정 시 헤더에 서버 정보가 노출되나, 별도 보안 설정 확인 (Tomcat SERVER.XML Path : $TOMCAT_SERVERXML_FILE)"
                    Evi=`sed '/<!--/,/-->/d' $TOMCAT_SERVERXML_FILE | awk '/<Connector /,/>/'`
                else
                    Result="Y"
                    Exp="Connector 태그의 SERVER 옵션이 설정되어 있으므로 양호함 (Tomcat SERVER.XML Path : $TOMCAT_SERVER_FILE)"
                    Evi=`sed '/<!--/,/-->/d' $TOMCAT_SERVERXML_FILE | awk '/<Connector /,/>/'`
                fi
            else
                Result="M"
                Exp="Tomcat 서비스가 실행하고 있으나, Server.xml 파일을 찾을 수 없으므로 수동 점검"
                Evi="TOMCAT_ROOT의 파일\n`ls -alR $TOMCAT_ROOT;ps auxwww | egrep 'catalina\.startup\.Bootstrap' | grep -v "grep"`
            fi
        else
            Result="Y"
            Exp="Apache 서비스 사용하지 않으므로 해당사항 없음"
            Evi=""
        fi

        Item_foot "$Result" "$Exp" "$Evi"
    }
}
```

• LINUX SRV-175 "NTP 및 시각 동기화 미설정" ShellScript 개선 전후 스크립트 실행 결과

항목 코드	스크립트 실행 결과 (24년도)	스크립트 실행 결과 (25년도)
LINUX <u>SRV-175</u>	<<u>자산명</u>> <u>자산명</u> </<u>자산명</u>> <<u>IP주소</u>> 172.16.X.X </<u>IP주소</u>> <<u>플랫폼</u>> OS </<u>플랫폼</u>> <<u>소프트웨어</u>> 리눅스 </<u>소프트웨어</u>> <<u>점검항목</u>> <u>SRV-175</u> </<u>점검항목</u>> <<u>점검항목명</u>> <u>NTP</u> 및 <u>시각 동기화</u> <u>미설정</u> </<u>점검항목명</u>> <<u>점검 결과</u>> M </<u>점검 결과</u>> <점검 내용> <u>NTP</u> : <u>ntpg-pn</u>을 통해 reach 값이 377에 가까운지 확인 필요 <u>Chrony</u> : <u>chronyc tracking</u>을 통해 stratum 값이 2 이하인지 확인 필요 [현황] <u>현황값 없음</u> </점검 내용>	<<u>자산명</u>> <u>자산명</u> </<u>자산명</u>> <<u>IP주소</u>> 172.16.X.X </<u>IP주소</u>> <<u>플랫폼</u>> OS </<u>플랫폼</u>> <<u>소프트웨어</u>> 리눅스 </<u>소프트웨어</u>> <<u>점검항목</u>> <u>SRV-175</u> </<u>점검항목</u>> <<u>점검항목명</u>> <u>NTP</u> 및 <u>시각 동기화</u> <u>미설정</u> </<u>점검항목명</u>> <<u>점검 결과</u>> N </<u>점검 결과</u>> <점검 내용> <u>Chrony</u> : <u>chronyc tracking</u>의 stratum 값이 2 초과이므로 취약함 [현황] Stratum : 4 </점검 내용>

• LINUX SRV-175 “NTP 및 시각 동기화 미설정” 스크립트 수정 내역

항목 코드	스크립트 수정 전 (24년도)	스크립트 수정 전 (25년도)
LINUX SRV-175	1) NTP & Chrony 서비스 동작 여부만 출력하여 수동 점검 필요 <pre>if [-f /etc/ntp.conf] then Evi_1=`ntpq -pn` # ntpq -pn은 NTP 동기화 피어 목록 출력 명령어 else Evi_1=`ntpq -pn` fi if [-f /etc/chrony.conf] then Evi_2=`chronyc tracking   grep -i stratum` else Evi_2=`chronyc tracking   grep -i stratum` fi Result="M" Exp="NTP 서버 동기화 설정 확인을 위해 NTP : ntpq-qn을 통해 reach 값이 377에 가까운지 확인 필요 \n Chrony : chronyc tracking 통해 stratum값이 2 이하인지 확인 필요"</pre>	1) NTP 값이 377 근처 & Chrony 값이 2이하가 아니면 [점검 결과] “취약” 출력하도록 수정 <pre>chrony_path=`rpm -ql chrony   grep "chrony.conf"   grep -v "gz"` ntpd_path=`rpm -ql ntp   grep "ntp.conf"   grep -v "gz"` if [-f \$chrony_path] then if [`chronyc tracking   sed -n 's/^Stratum\s*:\s*//p` -le 2] then Result="Y" Exp="Chrony : chronyc tracking을 통해 stratum값이 2이하이므로 양호함" Evi=`chronyc tracking   grep -i stratum` else Result="N" Exp="Chrony : chronyc tracking을 통해 stratum값이 2초과이므로 취약함" fi</pre>

항목 코드	스크립트 수정 전 (24년도)	스크립트 수정 전 (25년도)
LINUX SRV-175	1) NTP & Chrony 서비스 동작 여부만 출력하여 수동 점검 필요 <pre>if [-f /etc/ntp.conf] then Evi_1=`ntpq -pn` # ntpq -pn은 NTP 동기화 피어 목록 출력 명령어 else Evi_1=`ntpq -pn` fi if [-f /etc/chrony.conf] then Evi_2=`chronyc tracking   grep -i stratum` else Evi_2=`chronyc tracking   grep -i stratum` fi Result="M" Exp="NTP 서버 동기화 설정 확인을 위해 NTP : ntpq-qn을 통해 reach 값이 377에 가까운지 확인 필요 \n Chrony : chronyc tracking 통해 stratum값이 2 이하인지 확인 필요"</pre>	1) NTP 값이 377 근처 & Chrony 값이 2이하가 아니면 [점검 결과] “취약” 출력하도록 수정 <pre>chrony_path=`rpm -ql chrony   grep "chrony.conf"   grep -v "gz"` ntpd_path=`rpm -ql ntp   grep "ntp.conf"   grep -v "gz"` if [-f \$chrony_path] then if [`chronyc tracking   sed -n 's/^Stratum\s*:\s*//p` -le 2] then Result="Y" Exp="Chrony : chronyc tracking을 통해 stratum값이 2이하이므로 양호함" Evi=`chronyc tracking   grep -i stratum` else Result="N" Exp="Chrony : chronyc tracking을 통해 stratum값이 2초과이므로 취약함" fi</pre>

• LINUX SRV-175 “NTP 및 시각 동기화 미설정” 스크립트 비교

2024년도 Linux SRV-148 "웹 서비스 정보 노출" (Shellscript 개선 전)

```
NTP_synchronization() {
iCODE="SRV-175"
iTITLE="NTP 및 시각 동기화 미설정"
Item_head "$iCODE" "$iTITLE"

if [ -f /etc/ntp.conf ]
then
    Evi_1=`ntpq -pn` # ntpq -pn은 NTP 동기화 피어 목록 출력 명령어
else
    Evi_1=`ntpq -pn`
fi

if [ -f /etc/chrony.conf ]
then
    Evi_2=`chronyc tracking | grep -i stratum`
else
    Evi_2=`chronyc tracking | grep -i stratum`
fi

    Result="M"
    Exp="NTP 서버 동기화 설정 확인을 위해 NTP : ntpq-qn을 통해 reach 값이 377에 가까운
지 확인 필요 \n Chrony : chronyc tracking 통해 stratum값이 2 이하인지 확인 필요"
    Evi="$Evi_1" "$Evi_2"
    Item_foot "$Result" "$Exp" "$Evi"
}
```

Shell ▾

복사 캡션 ...

2025년도 Linux SRV-148 "웹 서비스 정보 노출" (Shellscript 개선 후)

```
NTP_synchronization() {
iCODE="SRV-175"
iTITLE="NTP 및 시각 동기화 미설정"
Item_head "$iCODE" "$iTITLE"

    chrony_path=`rpm -ql chrony | grep "chrony.conf" | grep -v "gz"`
    ntpd_path=`rpm -ql ntp | grep "ntp.conf" | grep -v "gz"`

    if [ `ps -ef | egrep -i "ntpd|chronyd" | grep -v "grep" | wc -l` -eq 0 ]
    then
        Result="N"
        Exp="NTP 서버와 동기화하고 있지 않으므로 취약함"
        Evi=`ps -ef | grep -i "ntpd|chronyd" | grep -v "grep"`
    else
        if [ -f $chrony_path ]
        then
            if [ `chronyc tracking | sed -n 's/^Stratum\s*:\s*//p` -le 2 ]
            then
                Result="Y"
                Exp="Chrony : chronyc tracking을 통해 stratum값이 2이하이므로 양호함"
                Evi=`chronyc tracking | grep -i stratum`
            else
                Result="N"
                Exp="Chrony : chronyc tracking을 통해 stratum값이 2초과이므로 취약함"
            fi
        elif [ -f $ntpd_path ]
        then
            Result="M"
            Exp="NTP 서버 동기화 설정 확인을 위해 NTP : ntpq-pn을 통해 reach값이 377에 가
까운지 확인 필요"
            Evi=`ntpq -pn`
        else
            if [ -f /etc/ntp.conf ]
            then
                Result="M"
                Exp="NTP 서버 동기화 설정 확인을 위해 NTP : ntpq-pn을 통해 reach 값이 377
에 가까운지 확인 필요"
            elif [ -f /etc/chrony.conf ]
            then
                if [ `chronyc tracking | sed -n 's/^Stratum\s*:\s*//p` -le 2 ]
                then
                    Result="Y"
                    Exp="Chrony: chronyc tracking을 통해 stratum값이 2이하이므로 양호함"
                    Evi=`chronyc tracking | grep -i stratum`
                else
                    Result="N"
                    Exp="Chrony: chronyc tracking을 통해 stratum값이 2초과이므로 취약함"
                    Evi=`chronyc tracking | grep -i stratum`
                fi
            else
                Result="M"
                Exp="NTP 서비스 실행중이나 설정파일이 없으므로 수동점검"
                Evi=`ps -ef | egrep "chronyd|ntpd" | grep -v "grep"`
            fi
        fi
    fi
    Item_foot "$Result" "$Exp" "$Evi"
}
```

• MySQL DBM-022 "데이터베이스의 주요 설정 파일, 패스워드 파일 등 주요 파일들의 접근 권한 적절성 여부" ShellScript 개선 전후 스크립트 실행 결과

항목 코드	스크립트 실행 결과 (24년도)	스크립트 실행 결과 (24년도)
MySQL DBM-022	<자산명> 자산명 </자산명> <IP주소> 172.16.X.X </IP주소> <플랫폼> DBMS </플랫폼> <소프트웨어> MySQL </소프트웨어> <점검항목> DBM-022 </점검항목> <점검항목명> 데이터베이스의 주요 설정 파일, 패스워드 파일 등 주요 파일들의 접근 권한 적절성 여부 </점검항목명> <점검 결과> N/A </점검 결과> <점검 내용> MySQL(MariaDB)는 해당사항 없음 [현황] 현황값 없음 </점검 내용>	<자산명> 자산명 </자산명> <IP주소> 172.16.X.X </IP주소> <플랫폼> DBMS </플랫폼> <소프트웨어> MySQL </소프트웨어> <점검항목> DBM-022 </점검항목> <점검항목명> 데이터베이스의 주요 설정 파일, 패스워드 파일 등 주요 파일들의 접근 권한 적절성 여부 </점검항목명> <점검 결과> M </점검 결과> <점검 내용> 수동 점검 [현황] -rw-r--r--. 1 root root 500 Apr 3 12:34 /etc/my.cnf drwxr-xr-x. 2 root root 4096 Apr 3 12:34 /etc/my.cnf.d -rw-r--r--. 1 root root 100 Apr 3 12:34 /etc/my.cnf.d/client.cnf -rw-----. 1 mysql mysql 600 Apr 3 12:34 /etc/mysql/mysql.conf.d/mysqld.cnf </점검 내용>

- MySQL DBM-022 “데이터베이스의 주요 설정 파일, 비밀번호 파일 등 주요 파일들의 접근 권한 적절성 여부” 스크립트 수정 내역

항목 코드	스크립트 수정 전 (24년도)	스크립트 수정 후 (25년도)
MySQL DBM-022	1) [현황]에 <u>my.cnf</u> & <u>my.cnf.d</u> 등 SQL 설정 파일의 접근 권한이 <u>미출력</u> - <u>Evi</u>=""	1) [현황]에 <u>my.cnf</u> & <u>my.cnf.d</u> 등 SQL 설정 파일의 접근 권한이 <u>출력</u> - <u>Evi</u>=`ls -alR /etc/   grep -i "my"`

MySQL DBM-022 "데이터베이스의 주요 설정 파일, 패스워드 파일 등 주요 파일들의 접근 권한 적절성 여부" 스크립트 비교	
# 2024년도 MySQL DBM-022 "데이터베이스의 주요 설정 파일, 패스워드 파일 등 주요 파일들의 접근 권한 적절성 여부" (Shellscript 개선 전)	# 2025년도 MySQL DBM-022 "데이터베이스의 주요 설정 파일, 패스워드 파일 등 주요 파일들의 접근 권한 적절성 여부" (Shellscript 개선 후)
<pre>fdb0603() { iCODE="DBM-022" iTITLE="데이터베이스의 주요 설정파일, 패스워드 파일 등 주요 파일들의 접근 권한 적절성 여부" Item_head "\$iCODE" "\$iTITLE" Result="N/A" Exp="MySQL(MariaDB)는 해당사항 없음" Evi="" Item_foot "\$Result" "\$Exp" "\$Evi" }</pre>	<pre>fdb0603() { iCODE="DBM-022" iTITLE="데이터베이스의 주요 설정파일, 패스워드 파일 등 주요 파일들의 접근 권한 적절성 여부" Item_head "\$iCODE" "\$iTITLE" Result="M" Exp="수동점검" Evi=`ls -alR /etc/   grep -i "my"` Item_foot "\$Result" "\$Exp" "\$Evi" }</pre>

- Kubernetes PRC-C-009 "시스템 주요 이벤트 로그 설정 미흡" 기초 정보

V플래그 값

로깅 레벨 (Verbosity Level) 설정 플래그

V플래그 값별 기능

- v=0 : 기본 정보 (양호)
- v=1~2 : 로그 정보 (양호)
- v=3~4 : 디버그 정보 (취약)
- v=5 : 트레이싱 정보 (취약)

조치 방안 : V플래그 값 설정 방법

kube-apiserver -v= <원하는 레벨값>

• Kubernetes PRC-C-009 “시스템 주요 이벤트 로그 설정 미흡” ShellScript 개선 전후 스크립트 실행 결과

항목 코드	스크립트 실행 결과 (24년도)	스크립트 실행 결과 (24년도)
<u>K8S</u> <u>PRC-C-</u> <u>009</u>	<<u>자산명</u>> <u>자산명</u> </<u>자산명</u>> <<u>IP주소</u>> 172.16.X.X </<u>IP주소</u>> <<u>플랫폼</u>> Container </<u>플랫폼</u>> <<u>소프트웨어</u>> <u>K8S Worker</u> </<u>소프트웨어</u>> <<u>점검항목</u>> PRC-C-009 </<u>점검항목</u>> <<u>점검항목명</u>> 시스템 주요 이벤트 로그 설정 미흡 </<u>점검항목명</u>> <점검 결과> N </점검 결과> <점검 내용> V 플래그가 3 이하으로 설정되어 있어 취약함 [현황] V = 2=2 =3 </점검 내용>	<<u>자산명</u>> <u>자산명</u> </<u>자산명</u>> <<u>IP주소</u>> 172.16.X.X </<u>IP주소</u>> <<u>플랫폼</u>> Container </<u>플랫폼</u>> <<u>소프트웨어</u>> <u>K8S Worker</u> </<u>소프트웨어</u>> <<u>점검항목</u>> PRC-C-009 </<u>점검항목</u>> <<u>점검항목명</u>> 시스템 주요 이벤트 로그 설정 미흡 </<u>점검항목명</u>> <점검 결과> Y </점검 결과> <점검 내용> V 플래그가 3 미만으로 설정되어 있어 양호함 V 플래그 : 시스템 주요 로그 수준 설정 [현황] V플래그 설정 현황 3 </점검 내용>

• Kubernetes PRC-C-009 “시스템 주요 이벤트 로그 설정 미흡” 스크립트 수정 내역

항목 코드	스크립트 수정 전 (24년도)	스크립트 수정 후 (25년도)
K8S PRC-C- 009	1) [점검 결과] 취약 기준 <u>미수정</u> - V플래그 값 3이하 취약 2) [현황] <u>출력값</u> <u>가독성</u> <u>미개선</u> - 스크립트 실행 시 'V =2=2 =3' 사용자 <u>비친화적인</u> 값 출력	1) [점검 결과] 취약 기준 수정 - V플래그 값 3미만 취약 1) [현황] <u>출력값</u> <u>가독성</u> 개선 - 현황 스크립트 'V'를 'V플래그 설정 <u>현황</u>'으로 개선 - 키(V플래그 설정 <u>현황</u>)과 값(3)를 <u>개행</u> 처리하여 출력

• Kubernetes PRC-C-009 “시스템 주요 이벤트 로그 설정 미흡” 스크립트 비교

2024년 PRC-C-009 "시스템 주요 이벤트 로그 설정 미흡" (Shellscript 개선 전)

```
PRC_C_009() {
    iCODE="PRC-C-009"
    iTitle="시스템 주요 이벤트 로그 설정 미흡"
    Item_head "$iCODE" "$iTitle"

    if [ "$check_k8s" = "k8s_Master" ];
    then
        k8s_logpath=$(grep -E "audit-log-path" "/etc/kubernetes/manifests/kube-apiserver.yaml"2>/dev/null)
        k8s_policy=$(grep -E "audit-policy-file" "/etc/kubernetes/manifests/kube-apiserver.yaml"2>/dev/null)

        if [[ -z "$k8s_logpath" && -z "$k8s_policy" ]];
        then
            Result="N"
            Exp="`audit-log-path`,`audit-policy-file'가 미설정 되어 있어 취약함"
            Evi="AuditLog Path=$k8s_logpath \nAuditlog Policy = $k8s_policy"

        elif [[ -z "$k8s_logpath" || -z "$k8s_policy" ]];
        then
            Result="N"
            Exp="`audit-log-path`,`audit-policy-file'가 미설정 되어 있어 취약함"
            Evi="AuditLog Path=$k8s_logpath \nAuditlog Policy = $k8s_policy"
        else
            Result="N"
            Exp="`audit-log-path`,`audit-policy-file'가 미설정 되어 있어 취약함"
            Evi="AuditLog Path=$k8s_logpath \nAuditlog Policy = $k8s_policy"
        fi

    elif [ "$check_k8s" = "k8s_Worker" ];
    then
        V_log=$(ps -ef | grep kubelet | awk -F '--v' '{split($2, arr, " "); print arr[1]}' | grep -v "grep")
        if [[ -n "$V_log" ]];
        then
            if [ "$V_log" -le 3 ];
            then
                Result="Y"
                Exp="V 플레그가 3이하로 설정되어 있어 양호함 \n V 플레그 : 시스템 주요 로
그 수준 설정"
                Evi="V = $V_log"
            else
                Result="N"
                Exp="V 플레그가 3초과로 설정되어 있어 취약함 (Default = 0) \n V 플레그 :
시스템 주요 로그 수준 설정"
                Evi="V = $V_log"
            fi
        else
            Result="N"
            Exp="V 플레그 설정값이 존재하지 않아 취약함 (Default = 0) \n V 플레그 : 시스
템 주요 로그 수준 설정"
            Evi="V = 설정값 미존재"
        fi
    fi
    Item_foot "$Result" "$Exp" "$Evi"
}
check_end "009"
```

Shell

복사캡션...

2025년 PRC-C-009 "시스템 주요 이벤트 로그 설정 미흡" (Shellscript 개선 후)

PRC_C_009() {
 iCODE="PRC-C-009"
 iTitle="시스템 주요 이벤트 로그 설정 미흡"
 Item_head "\$iCODE" "\$iTitle"

 if ["\$check_k8s" = "k8s_Master"];
 then
 k8s_logpath=\$(grep -E "audit-log-path" "/etc/kubernetes/manifests/kube-apiserver.yaml"2>/dev/null)
 k8s_policy=\$(grep -E "audit-policy-file" "/etc/kubernetes/manifests/kube-apiserver.yaml"2>/dev/null)

 if [[-z "\$k8s_logpath" && -z "\$k8s_policy"]];
 then
 Result="N"
 Exp="`audit-log-path`,`audit-policy-file'가 미설정 되어 있어 취약함"
 Evi="AuditLog Path=\$k8s_logpath \nAuditlog Policy = \$k8s_policy"

 elif [[-z "\$k8s_logpath" || -z "\$k8s_policy"]];
 then
 Result="N"
 Exp="`audit-log-path`,`audit-policy-file'가 미설정 되어 있어 취약함"
 Evi="AuditLog Path=\$k8s_logpath \nAuditlog Policy = \$k8s_policy"
 else
 Result="N"
 Exp="`audit-log-path`,`audit-policy-file'가 미설정 되어 있어 취약함"
 Evi="AuditLog Path=\$k8s_logpath \nAuditlog Policy = \$k8s_policy"
 fi

 elif ["\$check_k8s" = "k8s_Worker"];
 then
 V_log=\$(ps -ef | grep kubelet | grep -oP -- '--v=\K\d+ ' | uniq)
 for ilog in \${V_log[@]}
 do
 if [\$ilog -lt 3];
 then
 Result="Y"
 Exp="V 플레그가 3미만으로 설정되어 있어 양호함 (Default = 2) \n V 플레그 : 시
스템 주요 로그 수준 설정"
 Evi="V 설정 현황\n"\$V_log
 else
 Result="N"
 Exp="V 플레그가 3이상으로 설정되어 있어 취약함 \n V 플레그 : 시스템 주요 로그
수준 설정"
 Evi="V 설정 현황\n" \$V_log
 break
 fi
 done
 else
 Result="M"
 Exp="쿠버네티스 용도 확인 필요"
 Evi=""
 fi
 Item_foot "\$Result" "\$Exp" "\$Evi"
}
check_end "009"

- Kubernetes PRC-C-022 "레지스트리 연결 구간에 대한 보안 프로토콜 사용" 기초 정보

쿠버네티스에서 컨테이너 이미지 레지스트리 연결 구간이 HTTPS(보안 통신)를 사용 여부 점검

컨테이너 종류

- Containerd
- CRI-O

Containerd의 **Insecure** 설정 & CRI-O의 **Insecure_Registries** 설정

- 설정 : HTTP 통신/취약
- 미설정 : HTTPS 통신/양호

certs.d

- HTTPS 인증서 위치한 폴더

• Kubernetes PRC-C-022 "레지스트리 연결 구간에 대한 보안 프로토콜 사용" ShellScript 개선 전
후 스크립트 실행 결과

항목 코드	스크립트 실행 결과 (24년도)	스크립트 실행 결과 (24년도)
<u>K8S</u> <u>PRC-C-</u> <u>022</u>	24년도 현 항목 스크립트 <u>미존재</u>	<<u>자산명</u>> <u>자산명</u> </<u>자산명</u>> <<u>IP주소</u>> <u>172.16.X.X</u> </<u>IP주소</u>> <<u>플랫폼</u>> <u>Container</u> </<u>플랫폼</u>> <<u>소프트웨어</u>> <u>K8S Master</u> </<u>소프트웨어</u>> <<u>점검항목</u>> <u>PRC-C-022</u> </<u>점검항목</u>> <<u>점검항목명</u>> <u>레지스트리 연결 구간에 대한 보안 프로토콜 사용</u> </<u>점검항목명</u>> <<u>점검 결과</u>> <u>Y</u> </<u>점검 결과</u>> <<u>점검 내용</u>> <u>쿠버네티스 런타임 종류는 containerd.sock을 사용하고 있고, Config.toml 파일 내에 insecure 설정 존재하지 않아, HTTPS 통신 사용하고 있으므로 양호하며, Certs.d 폴더 내 인증서 존재하지 않음</u> [<u>현황</u>] <u>config.toml 파일 내 insecure 설정 현황</u> ## <u>template:true</u> <u># Use config version 2 to enable new configuration fields.</u> <u># Config file is parsed as version 1 by default.</u> <u>version = 2</u> <u>imports = ["/etc/containerd/conf.d/*.toml"]</u> <u>[plugins]</u> <u>[plugins."io.containerd.grpc.v1.cri"]</u> <u>sandbox_image = "<샌드박스 이미지 경로>"</u> <u>ignore_image_defined_volumes = true</u> <u>[plugins."io.containerd.grpc.v1.cri".containerd.runtimes.runc]</u> <u>runtime_type = "io.containerd.runc.v2"</u> <u>[plugins."io.containerd.grpc.v1.cri".containerd.runtimes.runc.options]</u> <u>SystemdCgroup = true</u> <u>Certs.d 폴더 내 인증서 현황</u> </<u>점검 내용</u>>

• Kubernetes PRC-C-022 "레지스트리 연결 구간에 대한 보안 프로토콜 사용" 스크립트 수정 내역

항목 코드	스크립트 수정 전 (24년도)	스크립트 수정 후 (25년도)
<u>K8S</u> <u>PRC-C-</u> <u>022</u>	24년도 평가 대상 누락	1) <u>Master/Worker Node</u>에 런타임 종류(<u>containerd/crio</u>)에 따라 <u>확인해야하는 설정 파일 불러오도록 추가</u> <pre><u>configtoml_path</u>="/etc/<u>containerd</u>/<u>config</u>/<u>toml</u>" <u>crioconf_path</u>="/etc/<u>crio</u>/<u>crio.conf</u>" if ["<u>\$sock_name</u>" = "<u>containerd.sock</u>"] then <u>insecure_configtoml</u>=`cat <u>\$configtoml_path</u>   grep -i "<u>*insecure*</u>"   grep -v "grep"` <u>elif</u> ["<u>\$sock_name</u>" = "<u>crio.sock</u>"] then <u>insecure_configtoml</u>=`cat <u>\$crioconf_path</u>   grep -i "<u>insecure_registries*</u>"   grep -v "grep"`</pre> 2) <u>Certd</u>. <u>폴더 경로</u>를 추가해서 <u>폴더 내 인증서 파일 유무 확인하도록 추가</u> <pre>if [`<u>\$insecure_configtoml</u>   wc -l` -eq 0] then <u>certs_path</u>="/etc/<u>containerd</u>/<u>certs.d</u>"</pre>

- Kubernetes PRC-C-022 "레지스트리 연결 구간에 대한 보안 프로토콜 사용" 코드 해석

컨테이너 런타임 종류 확인

1) Containerd 2) CRI-O

```
if [ "$sock_name" = "containerd.sock" ]  
elif [ "$sock_name" = "crio.sock" ]
```

노드 종류 확인

1) Master Node 2) Worker Node

```
if [ "$check_k8s" = "k8s_Master" ]
```

설정파일에서 Insecure 설정 여부 확인

1) Insecure 설정 : HTTP (취약) 2) Insecure 미설정 : HTTPS (양호)

```
insecure_configtoml=`cat $configtoml_path | grep -i "*insecure*" | grep -v "grep"`
```

Shell ▾복사캡션...

```
# 2025년 PRC-C-022 "레지스트리 연결 구간에 대한 보안 프로토콜 사용" (Shellscript)

PRC_C_022() {
    iCODE="PRC-C-022"
    iTitle="레지스트리 연결 구간에 대한 보안 프로토콜 사용"
    Item_head "$iCODE" "$iTitle"

    # 설정 파일의 경로 확인(고정)
    configtoml_path="/etc/containerd/config/toml"
    criocnf_path="/etc/crio/crio.conf"

    # 쿠버네티스 런타임 종류(containerd/crio) 확인
    sock_name=`cat $config | grep -oP '^[^/]+\\.sock$'`

    # 쿠버네티스 사용 용도(master/worker) 확인
    if [ "$check_k8s" = "k8s_Master" ]
    then
        if [ "$sock_name" = "containerd.sock" ]
        then
            insecure_configtoml=`cat $configtoml_path | grep -i "*insecure*" | grep -v "gre
rep"`

            #insecure 사용하지 않으면, certs.d 폴더에 인증서 사용 여부 확인
            if [ `insecure_configtoml | wc -l` eq 0 ]
            then
                certs_path="/etc/containerd/certs.d"
                if [ -d $certs_path ]
                then
                    Result="Y"
                    Exp="쿠버네티스 런타임 종류는 $sock_name을 사용하고 있고, Config.toml
파일 내에 insecure 설정이 존재하지 않아, HTTPS 통신 사용하고 있으므로 양호함"
                    Evi="config.toml 파일 내 insecure 설정 현황\n";$configtoml_path;`ech
o "Certs.d 폴더 내 인증서 현황\n";ls -alR $certs_path`
                else
                    Result="M"
                    Exp="쿠버네티스 런타임 종류는 $sock_name을 사용하고 있고, Config.toml
파일 내에 insecure 설정이 존재하지 않아, HTTPS 통신 사용하고 있으므로 양호하지만, Certs.d
폴더 내 인증서가 존재하지 않으므로 수동 점검"
                    Evi="config.toml 파일 내 insecure 설정 현황\n";$configtoml_path
                fi
            else
                Result="N"
                Exp="쿠버네티스 런타임 종류는 $sock_name을 사용하고 있고, Config.toml 파
일 내에 insecure 설정이 존재하여, HTTP 통신하고 있으므로 취약함"
                Evi="config.toml 파일 내 insecure 설정 현황\n";$configtoml_path;
            fi
        elif [ "$sock_name" = "crio.sock" ]
        then
            insecure_configtoml=`cat $criocnf_path | grep -i "insecure_registries*" | gr
ep -v "grep"`

            # insecure_registries 사용 여부 판단
            if [ `insecure_configtoml | wc -l` -eq 0 ]
            then
                Result="Y"
                Exp="쿠버네티스 런타임 종류는 $sock_name을 사용하고 있고, Config.toml
파일 내에 insecure 설정이 존재하지 않아, HTTPS 통신 사용하고 있으므로 양호함"
                Evi="config.toml 파일 내 insecure 설정 현황\n";$configtoml_path;`ech
o "Certs.d 폴더 내 인증서 현황\n";ls -alR $certs_path`
            else
                Result="N"
                Exp="쿠버네티스 런타임 종류는 $sock_name을 사용하고 있고, Config.toml 파
일 내에 insecure 설정이 존재하여, HTTP 통신하고 있으므로 취약함"
                Evi="config.toml 파일 내 insecure 설정 현황\n";$configtoml_path;
            fi
        elif [ "$check_k8s" = "k8s_Worker" ]
        then
            if [ "$sock_name" = "containerd.sock" ]
            then
                insecure_configtoml= `cat $configtoml_path | grep -i "*insecure*" | grep
-v "grep"`

                # insecure 사용하지 않으면, certs.d 폴더에 인증서 사용 여부 확인
                if [ `insecure_configtoml | wc -l` -eq 0 ]
                then
                    certs_path="/etc/containerd/certs.d"
                    if [ -d $certs_path ]
                    then
                        Result="Y"
                        Exp="쿠버네티스 런타임 종류는 $sock_name을 사용하고 있고, Config.t
oml 파일 내에 insecure 설정이 존재하지 않아, HTTPS 통신 사용하고 있으므로 양호함"
                        Evi="config.toml 파일 내 insecure 설정 현황\n";$configtoml_path;`
echo "Certs.d 폴더 내 인증서 현황\n";ls -alR $certs_path`
                    else
                        Result="M"
                        Exp="쿠버네티스 런타임 종류는 $sock_name을 사용하고 있고, Config.toml
파일 내에 insecure 설정이 존재하지 않아, HTTPS 통신 사용하고 있으므로 양호하지만, Certs.d
폴더 내 인증서가 존재하지 않으므로 수동 점검"
                        Evi="config.toml 파일 내 insecure 설정 현황\n";$configtoml_path
                    fi
                else
                    Result="N"
                    Exp="쿠버네티스 런타임 종류는 $sock_name을 사용하고 있고, Config.toml 파
일 내에 insecure 설정이 존재하여, HTTP 통신하고 있으므로 취약함"
                    Evi="config.toml 파일 내 insecure 설정 현황\n";$configtoml_path;
                fi
            elif [ "$sock_name" = "crio.sock" ]
            then
                insecure_configtoml=`cat $criocnf_path | grep -i "*insecure_registriesI" | g
rep -v "grep"`

                # insecure_registries 사용 여부 판단
                if [ `insecure_configtoml | wc -l` -eq 0 ]
                then
                    Result="Y"
                    Exp="쿠버네티스 런타임 종류는 $sock_name을 사용하고 있고, Config.toml 파
일 내에 insecure 설정이 존재하지 않아, HTTPS 통신 사용하고 있으므로 양호함"
                    Evi="config.toml 파일 내 insecure 설정 현황\n";$configtoml_path;`echo "Ce
rts.d 폴더 내 인증서 현황\n";ls -alR $certs_path`
                else
                    Result="N"
                    Exp="쿠버네티스 런타임 종류는 $sock_name을 사용하고 있고, Config.toml 파
일 내에 insecure 설정이 존재하여, HTTP 통신하고 있으므로 취약함"
                    Evi="config.toml 파일 내 insecure 설정 현황\n";$configtoml_path;
                fi
            else
                Result="M"
                Exp="쿠버네티스 런타임 종류는 $sock_name을 사용하고 있고, Config.toml 파일 내
에 insecure 설정이 존재하지 않아, HTTPS 통신 사용하고 있으므로 양호하지만, Certs.d 폴더 내
인증서가 존재하지 않으므로 수동 점검"
                Evi="config.toml 파일 내 insecure 설정 현황\n";$configtoml_path
            fi
        else
            Result="M"
            Exp="쿠버네티스 런타임 종류는 $sock_name을 사용하고 있고, Config.toml 파일 내에 in
secure 설정이 존재하지 않아, HTTPS 통신 사용하고 있으므로 양호하지만, Certs.d 폴더 내 인증
서가 존재하지 않으므로 수동 점검"
            Evi="config.toml 파일 내 insecure 설정 현황\n";$configtoml_path
        fi
    fi
    Item_foot"$Result" "$Exp" "$Evi"
}

check_end "022"
```

- Kubernetes PRC-C-024 "컨테이너간 불필요한 통신 제한 설정" 코드 해석

네트워크 정책(Network Policy) 설정하여 파드/네임스페이스 단위로 통신 접근 제어 여부 점검

파드(Pod) 복수 이상의 컨테이너의 묶음. 파드 내 컨테이너는 IP/Port/스토리지 일치

네임스페이스 파드 등의 리소스를 그룹화하는 가상 공간

- default
- kube-system 내부 네임스페이스
- kube-public 공개 네임스페이스

• Kubernetes PRC-C-024 “컨테이너간 불필요한 통신 제한 설정” ShellScript 개선 전후 스크립트 실행 결과

항목 코드	스크립트 실행 결과 (24년도)	스크립트 실행 결과 (24년도)
<u>K8S</u> <u>PRC-C-</u> <u>024</u>	24년도 현 항목 스크립트 <u>미존재</u>	<<u>자산명</u>> <u>자산명</u> </<u>자산명</u>> <<u>IP주소</u>> <u>172.16.X.X</u> </<u>IP주소</u>> <<u>플랫폼</u>> <u>Container</u> </<u>플랫폼</u>> <<u>소프트웨어</u>> <u>K8S Master</u> </<u>소프트웨어</u>> <<u>점검항목</u>> <u>PRC-C-024</u> </<u>점검항목</u>> <<u>점검항목명</u>> <u>컨테이너간 불필요한 통신 제한 설정</u> </<u>점검항목명</u>> <<u>점검 결과</u>> <u>N</u> </<u>점검 결과</u>> <<u>점검 내용</u>> <u>모든 네임스페이스에 networkpolicy가 정의되어 있지 않으므로 취약함</u> [현황] 1) <u>활성화된 namespace 목록 확인</u> NAME STATUS AGE default Active 365d <u>kube-node-lease</u> Active 365d <u>kube-public</u> Active 365d <u>kube-system</u> Active 365d 2) <u>활성화된 namespace 네트워크 정책 적용 여부 확인</u> NAMESPACE NAME POD-SELECTOR AGE infra <u>crong-kubenetes-event-exporter</u> <u>app.kubernetes.io</u> 365d </<u>점검 내용</u>>

• Kubernetes PRC-C-024 “컨테이너간 불필요한 통신 제한 설정” 스크립트 수정 내역

항목 코드	스크립트 수정 전 (24년도)	스크립트 수정 후 (25년도)
<u>K8S</u> PRC-C-024	24년도 평가 대상 누락	1) 활성화된 <u>Namespace</u>를 확인하여 네트워크 정책 적용했는지 확인하는 코드 추가 <pre>for <u>ns_active</u> in \$(<u>kubectl</u> get <u>ns</u> --no-headers <u>awk</u> '\$2 == "Active" (print \$1)') count=\$(<u>kubectl</u> get <u>networkpolicy</u> -n "<u>\$ns</u>" --no-headers 2>/dev/null <u>wc</u> -l)</pre>

- Kubernetes PRC-C-024 "컨테이너간 불필요한 통신 제한 설정" 코드 해석

노드 종류 확인

1) Master Node 2) Worker Node

```
if [ "$check_k8s" = "k8s_Master" ]
```

네임스페이스 활성화 여부 확인

```
for ns_active in $(kubectl get ns --no-headers | awk '$2 == "Active" {print $1}')
```

네임스페이스별 네트워크정책 확인

```
count=$(kubectl get networkpolicy -n "$ns" --no-headers 2>/dev/null | wc -l)
```

- Kubernetes PRC-C-024 "컨테이너간 불필요한 통신 제한 설정" 스크립트

2025년 PRC-C-024 "컨테이너간 불필요한 통신 제한 설정 (Master Node)" (Shellscript)

```
PRC_C_024() {
    iCODE="PRC-C-024"
    iTitle="컨테이너간 불필요한 통신 제한 설정"
    Item_head "$iCODE" "$iTitle"

    if [ "$check_k8s" = "k8s_Master" ]
    then
        # Namespace 활성화 여부 확인
        for ns_active in $(kubectl get ns --no-headers | awk '$2 == "Active" {print $1}')
        do
            # for문으로 각 네임스페이스 policy 정책 확인하도록 구성
            count=$(kubectl get networkpolicy -n "$ns" --no-headers 2>/dev/null | wc -l)
            if [ "$count" -eq 0 ]
            then
                Result="Y"
                Exp="모든 네임스페이스에 networkpolicy가 정의되어 있으므로 양호함"
                Evi="활성화된 namespace 목록 확인\n``kubectl get namespace`\n\n활성화된 namespace 네트워크 정책 적용 여부 확인\n``kubectl get networkpolicy --all-namespaces`"
            else
                Result="N"
                Exp="모든 네임스페이스에 networkpolicy가 정의되어 있지 않으므로 취약함"
                Evi="활성화된 namespace 목록 확인\n``kubectl get namespace`\n\n활성화된 namespace 네트워크 정책 적용 여부 확인\n``kubectl get networkpolicy --all-namespaces`"
            fi
            Item_foot "$Result" "$Exp" "$Evi"
        }
    check_end "024"
```

- Kubernetes PRC-C-049 "PID 제한 설정 (Worker Node)" 코드 해석

컨테이너(Pod)가 수용 한계 이상 프로세스 생성으로 시스템 리소스 고갈 방지 위한 보안 설정 여부 확인

`PodPidsLimit` PID 제한 설정 옵션

- 제한 없음 (취약)
- -1 (취약)

• Kubernetes PRC-C-049 "PID 제한 설정 (Worker Node)" ShellScript 개선 전후 스크립트 실행 결과

항목 코드	스크립트 실행 결과 (24년도)	스크립트 실행 결과 (24년도)
<u>K8S</u> <u>PRC-C-</u> <u>049</u>	24년도 현 항목 스크립트 <u>미존재</u>	<<u>자산명</u>> <u>자산명</u> </<u>자산명</u>> <<u>IP주소</u>> 172.16.X.X </<u>IP주소</u>> <<u>플랫폼</u>> Container </<u>플랫폼</u>> <<u>소프트웨어</u>> <u>K8S</u> Worker </<u>소프트웨어</u>> <<u>점검항목</u>> PRC-C-049 </<u>점검항목</u>> <<u>점검항목명</u>> <u>PID</u> <u>제한</u> <u>설정</u> </<u>점검항목명</u>> <<u>점검 결과</u>> <u>N</u> </<u>점검 결과</u>> <<u>점검 내용</u>> <u>Pod 실행되는 프로세스를 제한하고 있지 않으므로 취약함 (Default Value : -1)</u> [현황] kind: <u>KubeletConfiguration</u> apiVersion: <u>kubelet.config.k8s.io/v1beta1</u> address: 0.0.0.0 port: 10250 authentication: anonymous: enabled: false <u>webhook:</u> enabled: true <u>x509:</u> <u>clientCAFile:</u> "/etc/<u>kubernetes</u>/<u>pki</u>/<u>ca.crt</u>" authorization: mode: <u>Webhook</u> <u>clusterDomain:</u> "cluster.local" <u>clusterDNS:</u> - "10.96.0.10" <u>failSwapOn:</u> false <u>cgroupDriver:</u> "systemd" <u>runtimeRequestTimeout:</u> "15m" <u>PodPidsLimit:</u> -1 </<u>점검 내용</u>>

• Kubernetes PRC-C-049 "PID 제한 설정 (Worker Node)" 스크립트 수정 내역

항목 코드	스크립트 수정 전 (24년도)	스크립트 수정 후 (25년도)
<u>K8S</u> <u>PRC-C-</u> <u>049</u>	24년도 평가 대상 누락	1) <u>PodPidsLimit</u> 값에 따른 점검 결과 출력 <pre>pid_limit_value=`grep -oP '<u>PodPidsLimit</u>[= ]\K[0-9]+' \$config` if [` \$config   grep -i "<u>PodPidsLimit</u>"   wc -l ` -eq 0] [<u>\$pid_limit_value</u> -eq -1] then Result="N" Exp="Pod 실행되는 프로세스를 제한하고 있지 않으므로 취약함 (Default Value : -1)" <u>Evi</u>=`cat \$config` fi else if [<u>ps -ef</u> grep -v "grep" grep "--pod-max-pids" wc -l ` -eq 0 ]    [ <u>ps -ef</u>   grep -v "grep"   grep "--pod-pids-limit"   wc -l ` -eq 0] then Result="N" Exp="Pod 실행되는 프로세스를 제한하고 있지 않으므로 취약함" <u>Evi</u>=`cat \$config:echo "\nPID 프로세스 현황";ps -ef   grep -v "grep"   grep - E "--pod-max-pids --pod-pids-limit"`</pre>

- Kubernetes PRC-C-049 "PID 제한 설정 (Worker Node)" 코드 해석

노드 종류 확인 (해당 항목 점검 대상 : Worker Node)

```
elif [ "$check_k8s" = "k8s_Worker" ]  
then  
    if [ -f $config ]
```

Config 파일 내 PodPidsLimit 옵션 확인

```
pid_limit_value=`grep -oP 'PodPidsLimit[= ]\K[0-9]+' $config`
```

PodPidsLimit 값에 따른 점검 결과 출력

```
if [ ` $config | grep -i "PodPidsLimit" | wc -l ` -eq 0 ] || [ $pid_limit_value -eq -  
1 ]  
then  
    Result="N"  
    Exp="Pod 실행되는 프로세스를 제한하고 있지 않으므로 취약함 (Default Value : -1)"  
    Evi=`cat $config`
```

- Kubernetes PRC-C-049 "PID 제한 설정 (Worker Node)" 스크립트

```
# 2025년 PRC-C-049 "PID 제한 설정 (Worker Node)" (Shellscript)
```

```
PRC_C_049() {
    iCODE="PRC-C-049"
    iTitle="PID 제한 설정"
    item_head "$iCODE" "iTitle"
    if [ "$check_k8s" = "k8s_Master" ]
    then
        Result="N/A"
        Exp="Master Node에서는 해당사항 없음"
        Evi=""
    elif [ "$check_k8s" = "k8s_Worker" ]
    then
        if [ -f $config ]
        then
            pid_limit_value=`grep -oP 'PodPidsLimit[= ]\K[0-9]+' $config`
            if [ ` $config | grep -i "PodPidsLimit" | wc -l ` -eq 0 ] || [ $pid_limit_valu
e -eq -1 ]
            then
                Result="N"
                Exp="Pod 실행되는 프로세스를 제한하고 있지 않으므로 취약함 (Default Value
: -1)"
                Evi=`cat $config`
            fi
        else
            if [ `ps -ef | grep -v "grep" | grep "--pod-max-pids" | wc -l ` -eq 0 ] || [ `
ps -ef | grep -v "grep" | grep "--pod-pids-limit" | wc -l ` -eq 0 ]
            then
                Result="N"
                Exp="Pod 실행되는 프로세스를 제한하고 있지 않으므로 취약함"
                Evi=`cat $config;echo "\nPID 프로세스 현황";ps -ef | grep -v "grep" | gre
p -E "--pod-max-pids|--pod-pids-limit"`
            else
                Result="Y"
                Exp="Pod 실행되는 프로세스를 제한하고 있으므로 양호함"
                Evi=`cat $config | grep -i "PodPidsLimit";echo "\nPID 프로세스 현황";ps -
ef | grep -v "grep" | grep -E "--pod-max-pids|--pod-pids-limit"`
            fi
        fi
    else
        Result="M"
        Exp="쿠버네티스 용도 확인 필요"
        Evi=""
    fi
    Item_foot "$Result" "$Exp" "$Evi"
}
check_end "049"
```

- Docker PRC-C-024 "Default Network Bridge 내 네트워크 트래픽 제한 설정" 코드 해석

Docker Default Network Bridge에서 불필요한 컨테이너 간 통신(icc; inter-container communication) 허용 여부 점검

icc

- true (업무상 불필요한 컨테이너인지 인터뷰)
- false (양호)

daemon.json

icc 옵션 미설정시 daemon.json 파일에서 daemon_icc값으로 점검

- daemon_icc=true (업무상 불필요한 컨테이너인지 인터뷰)
- daemon_icc=false (양호)

• Docker PRC-C-024 “Default Network Bridge 내 네트워크 트래픽 제한 설정” ShellScript 개선 전
후 스크립트 실행 결과

항목 코드	스크립트 실행 결과 (24년도)	스크립트 실행 결과 (24년도)
Docker PRC-C- 024	<<u>자산명</u>> <u>자산명</u> </<u>자산명</u>> <<u>IP주소</u>> 172.16.X.X </<u>IP주소</u>> <<u>플랫폼</u>> Container </<u>플랫폼</u>> <<u>소프트웨어</u>> Docker </<u>소프트웨어</u>> <<u>점검항목</u>> PRC-C-024 </<u>점검항목</u>> <<u>점검항목명</u>> Default Network Bridge 내 네트워크 트래픽 제한 설정 </<u>점검항목명</u>> <<u>점검 결과</u>> M </<u>점검 결과</u>> <점검 내용> <u>icc</u> 옵션 확인할 수 없으므로 수동 점검 [현황] <u>현황값 없음</u> </점검 내용>	<<u>자산명</u>> <u>자산명</u> </<u>자산명</u>> <<u>IP주소</u>> 172.16.X.X </<u>IP주소</u>> <<u>플랫폼</u>> Container </<u>플랫폼</u>> <<u>소프트웨어</u>> Docker </<u>소프트웨어</u>> <<u>점검항목</u>> PRC-C-024 </<u>점검항목</u>> <<u>점검항목명</u>> Default Network Bridge 내 네트워크 트래픽 제한 설정 </<u>점검</u> <u>항목명</u>> <<u>점검 결과</u>> N </<u>점검 결과</u>> <점검 내용> <u>Daemon.json</u> 설정 파일 내 <u>icc</u> 값이 <u>true</u>로 설정되어 있으므로 취약함 [현황] <u>Daemon_icc=true</u> </점검 내용>

• Docker PRC-C-024 "Default Network Bridge 내 네트워크 트래픽 제한 설정" 스크립트 수정 내역

항목 코드	스크립트 수정 전 (24년도)	스크립트 수정 후 (25년도)
Docker PRC-C-024	1) Docker <u>Daemon.json</u> 파일 내 <u>icc</u>값 <u>미점검</u> -	1) Docker <u>Daemon.json</u> 파일 내 <u>icc</u>값 <u>점검</u> <pre>else if [["<u>\$daemon_icc</u>" == false]]; then <u>flag1=Y</u> <u>Evi="Daemon.json 설정 파일 내 icc 값이 false로 설정되어 있으므로 양호함"</u> else <u>flag1=N</u> <u>Evi="icc 옵션을 사용하지 않을 시 Default 설정값으로 사용하므로 취약함 (Default : True)"</u></pre>

- Docker PRC-C-024 "Default Network Bridge 내 네트워크 트래픽 제한 설정" 스크립트

2024년 PRC-C-024 "Default Network Bridge 내 네트워크 트래픽 제한 설정" (Shellscript)

```
Evidence_PRC_C_024() {
    echo "["icc" 사용 여부]"
    echo "-----"
    a1=`docker network ls --quiet | xargs docker network inspect --format '{{.Name}}: {{.Options}}' | awk -F 'com.docker.network.bridge.enable_icc:' '{print $2}' | awk -F ' com' '{print $1}' | uniq -f 2`
    echo "com.docker.network.bridge.enable_icc: $a1"
    if [ "$a1"=="false" ];
    then
        flag1=Y
        Evi="icc를 사용하지 않으므로 양호함"
    elif [ "$a1"=="true" ];
    then
        flag1=N
        Evi="icc를 사용하여 취약함(Default:true)"
    else
        flag1=M
        Evi="icc 옵션 확인할 수 없으므로 수동점검"
    fi
}
```

Shell ▾

복사 캡션 ...

2025년 PRC-C-024 "Default Network Bridge 내 네트워크 트래픽 제한 설정" (Shellscript)

```
Evidence_PRC_C_024() {
    echo "["icc" 사용 여부]"
    echo "-----"
    a1=`docker network ls --quiet | xargs docker network inspect --format '{{.Name}}: {{.Options}}' | awk -F 'com.docker.network.bridge.enable_icc:' '{print $2}' | awk -F ' com' '{print $1}' | uniq -f 2`
    echo "com.docker.network.bridge.enable_icc: $a1"
    if [ "$a1"=="false" ];
    then
        flag1=Y
        Evi="icc를 사용하지 않으므로 양호함"
    elif [ "$a1"=="true" ];
    then
        flag1=N
        Evi="icc를 사용하여 취약함(Default:true)"
    else
        if [[ "$daemon_icc"=="false" ]];
        then
            flag1=Y
            Evi="Daemon.json 설정 파일 내 icc 값이 false로 설정되어 있으므로 양호함"
        else
            flag1=N
            Evi="icc 옵션을 사용하지 않을 시 Default 설정값으로 사용하므로 취약함 (Default : True)"
        fi
    fi
}
```